31 August 2026

Proposal for

IndiGo ITSM Automation RFP

AionOS

Submitted to	InterGlobe Aviation Limited (IndiGo)
Submitted by	AIONOS Inc.
Date	31 October 2026 (Revised Ver1.4)
Classification	**CONFIDENTIAL: Authorized Recipients Only**

Contents

1. Executive Summary 6
   Solution Themes 7
2. Our Understanding of IndiGo's Objectives 8
   Business Drivers and our understanding of them. 9
   Confirming the Boundary of the Ask 11
3. Proposed Solution Overview 11
   RUN: Transition and Operate Plan for Website and Mobile 12
   Change: Automation Activation Plan for Website and Mobile 14
   Where DevRev Fits, and Where It Doesn't 15
   Roles and Responsibilities Across RUN and Change 16
4. Agentic AI and ITSM Automation Capability 17
   Mapped to the RFP's Expected Business Outcomes 17
   How an Incident Moves Through the AI Layer 18
   Explainability, Override, and Rollback 20
   DevRev Reference Architecture 21
      How This Works 22
5. 24x7 Production Support and Incident Management Operating Model 22
   Coverage Model: 24x7x365 23
   The Incident Lifecycle 24
   Priority-Based SLA Framework 25
   P1 War-Room Protocol 26
   Operational Escalation Path 26
   Code-Level Debugging and Access 26
   Shift Handover and Continuity 27
   Proactive Health Checks 27
6. Tooling, Integration and Technical Architecture 27
   Integration Architecture 27
   Technology Stack at a Glance 28
   Integration Points in Detail 29
7. Non-Functional Requirements Compliance 30
   Scope Boundary: Support Service vs. Application Infrastructure 30
8. Digital Performance Management and Observability 32

24x7 Application Monitoring Team 32
What the Monitoring Team Does 33
What Every Ticket Contains 33
Coverage and Staffing 33
Boundaries 33
Instrumentation: Confirmed Tools, Across Every Channel 34
Metrics Tracked, End to End 34
Correlating Frontend Symptoms with Backend Root Cause 35
Proactive Detection, Before Customer Impact 35
Continuous Optimization of Customer Experience 35
9. Problem Management, RCA and Continuous Improvement 35
From Recurring Incident to Closed Problem 35
Known Error Database: What Gets Captured 36
Recurrence Reduction: How We Measure This Is Working 37
Correlating NPS with Production Incidents 37
10. Security, Data Privacy and Compliance 37
Access Control and Authentication 37
Encryption and Environment Isolation 37
PII Masking 37
Data Privacy Regulations 38
PCI-DSS Posture 38
11. SLA, Governance, and the Target Operating Model 38
Delivery Location 38
Program Governance and Escalation 38
Review Cadence 39
Executive Dashboards and Reporting 39
12. Team Structure and Talent Model 39
By Day of Week and Hour 40
By Channel 40
Roles, Skills, and Experience 42
Automation Potential Across the Ticket Base 43
Login, OTP & Authentication (170 tickets, ~13/month) 44
Check-in, Boarding & Seat (298 tickets, ~23/month) 45
Baggage (199 tickets, ~15/month) 45
Payment & Billing (376 tickets, ~29/month, Medium-High) 45

Loyalty (BluChip) & Rewards (92 tickets, ~7/month, Medium-High) 46
Team Efficiency as Automation Matures 46
Baseline: The 26-Person Structure 46
What Automation Changes, and What It Doesn't 47
The Reduction, Role by Role 47
Role-by-Role Rationale 47
Rollout Timeline 48
13. Product Roadmap 49
Why These Three, and Not Others 49
14. Relevant Experience, Case Studies and Credentials 49
Case Study – 1 49
Case Studies – 2 51
Case Study – 3 51
15. Compliance Matrix (Appendix A) 55
Functional Requirements (3.1 to 3.13) 55
Non-Functional Requirements (4.1 to 4.13) 60
16. Assumptions and Exceptions (Appendix B) 61
Assumptions 61
Exceptions 63
17. Commercial Proposal – By Cost Head (3-Year View) 64
17.1 Automation cost summary (Cost included above in commercial proposal) ..... **Error! Bookmark not defined.**

1. Executive Summary

IndiGo's direct digital channels, the website and the mobile app, carry the transactions that matter most to the business: bookings, payments, check-in, and boarding. This RFP asks for a partner who can stabilize production support for these channels today while building toward the agentic AI-led automation IndiGo names repeatedly as its ambition. We propose a clean sequence into a parallel run: **RUN**'s 8-week transition (Exhibit 8) takes over and stabilizes operations first. Once **RUN** reaches steady-state and Day 1 begins, a separate technical team starts **Change** in parallel with it, building on a DevRev instance for triage and RCA. **Change's** platform setup starts right away. Its domain-specific work, building the knowledge base and validating AI triage from real incidents, starts only once RUN is live and generating that data.

RUN takes over production support from the incumbent team across an 8-week transition, then operates business as usual from Day 1 onward, against priority-based SLAs agreed with IndiGo during Transition. **Change** runs alongside RUN from that same Day 1: a separate team integrates DevRev and reconciles the observability stack first, using a pattern already proven on IndiGo's Payment triage and RCA, then builds the knowledge base from RUN's real incidents, validates AI triage against them, and activates full workflow automation. Change's technical setup starts on Day 1, alongside RUN; its domain-specific stages start only once RUN's steady-state operations are generating the real incident data they depend on. Nothing here runs on an assumption; every data-dependent stage inherits evidence from the one before it.

24x7 Production support, independent coverage from Day 1	**Seconds** To classify and triage; our endeavor is to reduce MTTR as the automation layer matures	**6** Months to full automation from Day 1, once RUN's 8-week transition concludes and RUN and Change begin together

Exhibit 1. What we are committing to deliver, at a glance.

These three commitments are not separate promises. RUN and Change carry theirs simultaneously, and the diagram below shows how each one carries its share of the ask.

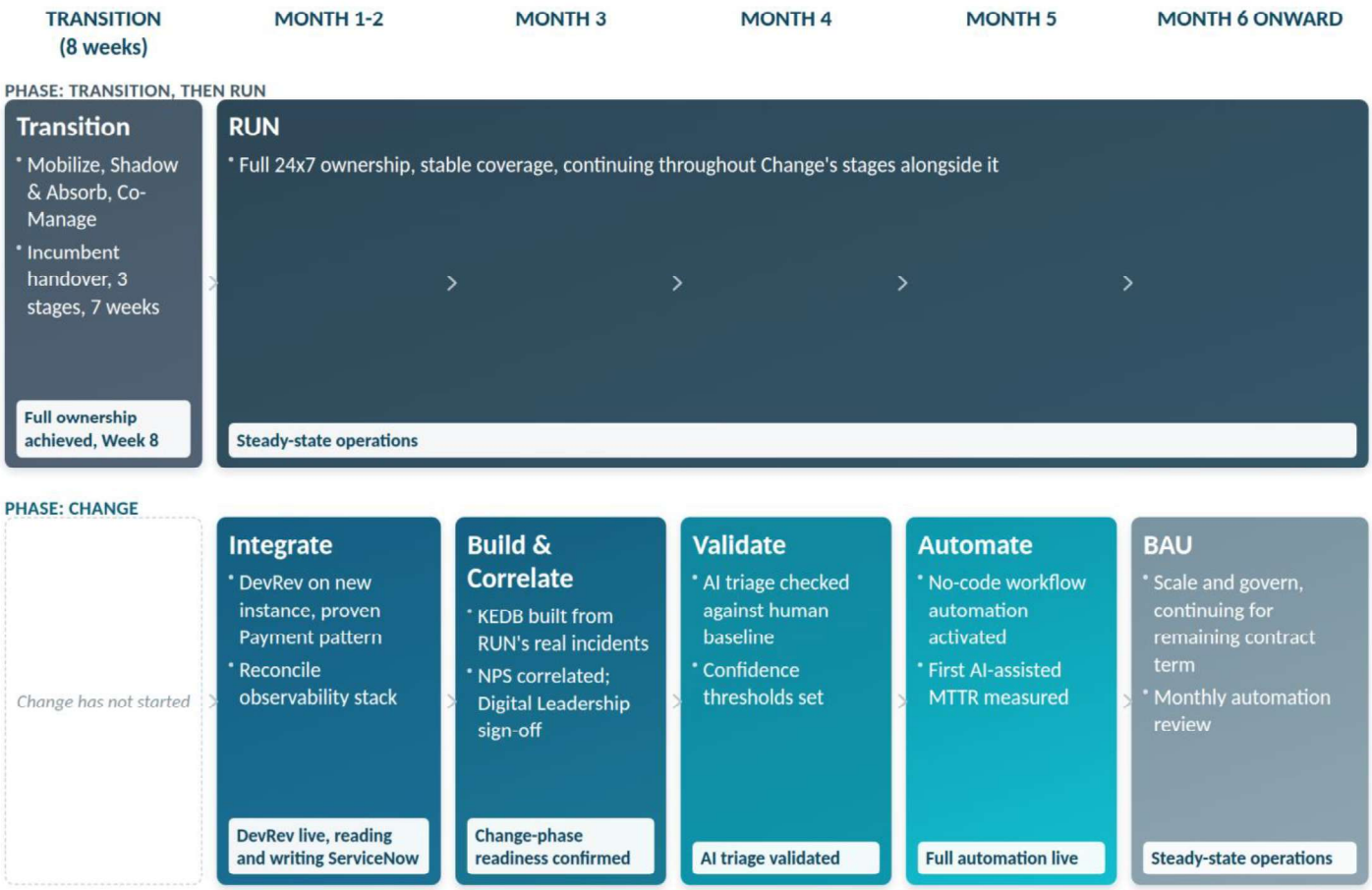


Exhibit 2. Transition completes first; RUN and Change then run in parallel from that same Day 1.

Transition (8 weeks) is IndiGo's incumbent handover, covered in detail in Exhibit 7 and Exhibit 8. RUN and Change both begin on Day 1, the day Transition concludes and run in parallel from that point through the remainder of the engagement.

Solution Themes

Solution Theme	Why It Matters to IndiGo
Two dedicated teams, no shared bandwidth	RUN's team focuses entirely on production support; Change's technical team focuses entirely on integration and automation. Running in parallel from Day 1 does not mean competing for the same attention, sign-off cycles, or governance bandwidth, since neither team is asked to do the other's job.
Governance with a named, time-bound authority	RCA and SLA disputes escalate through a defined matrix to IndiGo Digital Leadership.

Solution Theme	Why It Matters to IndiGo
Performance degradation treated as seriously as an outage	Reflects the RFP's own instruction that slows down for a booking and payment platform.
A proven foundation before automation	DevRev's technical setup starts on Day 1. What still waits is domain-specific: AI triage is validated against real websites and mobile incidents only once RUN has generated that data, in Month 4. Automation launches evidence for this specific domain.
Tribal knowledge of IndiGo through an existing relationship	A new vendor spends the first several months of any engagement learning what your own team already takes for granted, who owns which system, which workarounds exist for a reason, which stakeholders need to be looped in before a change goes out, and how IndiGo's own escalation culture actually works in practice versus on paper. AIONOS skips that curve entirely: as a partner already engaged with IndiGo, we bring working familiarity with your environment, your teams, and your ways of working from day one, so the Mobilize and Shadow stages in Section 3 genuinely compress in practice.

Exhibit 3. Solution themes

2. Our Understanding of IndiGo's Objectives

IndiGo is India's largest domestic carrier, running a low-cost model across more than 400 aircraft and 114 destinations. Its website and mobile app are not support channels sitting alongside the business; for a huge share of IndiGo's customers, they are the point of sale. A booking flow that fails or slows down does not just generate a support ticket, it costs a fare. We have read this RFP with that context in view, and it shapes how we prioritize everything that follows.

Exhibit 4. Our understanding of IndiGo's objectives, briefly.

Business Drivers and our understanding of them.

Business Driver	What We Read into It
Improved Production Stability and Availability	An outage or slowdown on the booking and payment path is a revenue event first and a technical incident second. We prioritize protecting conversion as much as uptime.
Reduction in Incident Resolution Time	IndiGo has confirmed its own current baseline is approximately 7 days; a 40% improvement against that baseline works out to roughly 4.2 days. However, the scope of the RFP is RCA and Triage is not Incident Resolution (L3 team). We acknowledge that RCA and Triage will enable this metric and are happy

Business Driver	What We Read into It
	to share this responsibility with the Product teams handling L3 and actual rectification.
24x7 Operational Coverage	Coverage has to match how the airline operates, including peak travel periods such as Diwali, summer, and long weekends. We acknowledge this.
Proactive Incident Prevention	Shifting from reactive triage to prevention depends on catching degradation before it becomes an incident: Section 8's Smart Detection flags anomalies as they develop, and DevRev's Detect and Correlate capability (Section 4) groups the related signals into one actionable alert before RUN must respond to a full-blown incident.
Enhanced Transparency and Stakeholder Communication	IndiGo's Digital Leadership needs real-time visibility into incident status, beyond what a monthly report can offer; this shapes our governance and dashboard approach directly.
Observability for Web and Mobile	Observability for web and mobile is called out in the RFP, but Grafana is IndiGo's existing APM tool and is already part of the current monitoring setup as answered in the queries. Since this platform is already in use within the IndiGo ecosystem, AIONOS is not proposing a separate solution here.
Standardization of Incident and Problem Management	IndiGo wants ITIL-aligned discipline built on tools it already owns, ServiceNow and JIRA, rather than a parallel process the partner introduces on top.
Reduced Recurrence of Production Issues	A recurring incident is a signal the first fix did not address the root cause. We treat repeat incidents as a problem-management failure.
Operational Efficiency Through Automation	The RFP's own vendor capability table treats this as a preference rather than a mandate. We read this as license to prove automation against real incidents before fully trusting its output, not a mandate to delay starting it: DevRev's technical setup runs from Day 1, and validation against RUN's real data follows once RUN is live.

Business Driver	What We Read into It
Support for Business Growth and Digital Initiatives	The support model has to scale with transaction volume and new digital products without a linear increase in cost or headcount, since that scalability is what makes the RUN and Change journey viable as IndiGo grows.
Delivery and Sprint Alignment	Production issues cannot sit in a support silo; the RFP expects our team in IndiGo's solutioning calls and sprint cycles, feeding defects back before they become recurring incidents.
Improved Business Continuity and Customer Experience	NPS and customer trust are the actual scoreboard. Every other driver in this table ultimately serves this one.

Exhibit 5. Driver that motivate the solution.

Two important drivers, automation and business continuity, get their own dedicated treatment later in our response: Section 4 covers the agentic AI and automation capability directly, and Section 9 covers how problem management and RCA reduce recurrence over time.

Confirming the Boundary of the Ask

Some parts of the RFP are explicit. Others use language, such as "not limited to website and app," that leaves room for interpretation. We have separated the two below rather than quietly assume our way past the gaps.

In Scope, As Stated in the RFP	Our Understanding
Website (desktop and mobile web) and mobile applications (iOS and Android)	Website (desktop and mobile web B2C & B2B) and the React native mobile apps (B2B & B2C).
IndiGo's existing ServiceNow instance, integrated with JIRA for delivery handoff.	We assume that AIONOS will triage/RCA and hand off to the development PODs for L3.
24x7 Monitoring and Triage, RCA staffed 16x5 with on-call cover for P1 and P2, and priority-based SLAs agreed during Transition, with defined escalation.	IndiGo has a final say when an RCA finding, and an SLA priority decision are disputed at the same time.

Exhibit 6. What the RFP states, and what we understand it to mean where the RFP itself leaves room for interpretation.

The remainder of this proposal is structured to answer those questions directly, beyond the RFP's own compliance checklist.

3. Proposed Solution Overview

This proposal is not a technology stack. It is an operating plan with two phases, RUN and Change, each with its own timeline, activities, and deliverables.

The sections below cover each phase in turn, starting with RUN, which begins on Day 1.

RUN: Transition and Operate Plan for Website and Mobile

Transition runs in three stages over eight weeks, from initial access and governance through to assisted co-management with the incumbent; standalone 24x7 ownership is what RUN phase itself achieves once Transition concludes. In Co-Manage, the team runs live incidents alongside incumbent backstop before taking full ownership. Each stage has a clear duration, and a specific deliverable IndiGo can check against. Once Transition concludes and RUN begins, a separate team starts Change's Integrate stage the same day, covered next; RUN's own team is dedicated to production support throughout.

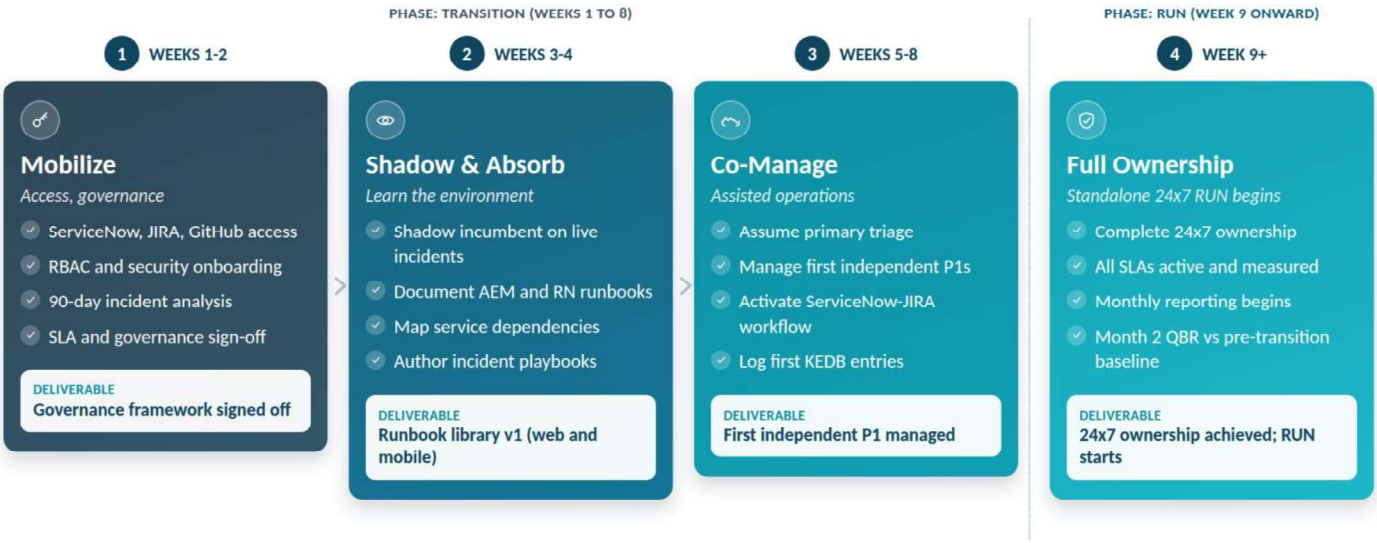


Exhibit 7. Transition's three stages across eight weeks, then RUN begins with standalone ownership.

Stage	Duration	Key Activities	Deliverable
Mobilize	Weeks 1 to 2	Provision ServiceNow, JIRA, GitHub, AEM, and Firebase access; complete RBAC and security onboarding; run a 90-day historical incident analysis; finalize the SLA matrix and governance sign-off.	Governance framework signed off
Shadow and Absorb	Weeks 3 to 4	Shadow the incumbent team across live web and mobile incidents; document runbooks for AEM and React Native (RN); map service dependencies against live traffic; author critical incident playbooks.	Runbook library v1 (web and mobile)
Co-Manage	Weeks 5 to 8	Assume primary triage with incumbent backstop oversight; manage the first independent P1 incidents under IndiGo oversight; activate ServiceNow to JIRA workflow; log first KEDB entries.	First independent P1 managed
TRANSITION ENDS — RUN BEGINS			
Full Ownership — RUN begins	Week 9 onward	Complete 24x7 standalone ownership as the incumbent stands down; all P1 to P4 SLAs active and measured; monthly service health reporting begins; Month 2 QBR reviews MTTR against the pre-transition (incumbent) baseline.	24x7 ownership achieved; RUN starts

Exhibit 8. RUN transition stages, activities, and deliverables in detail.

The same three stages, shown as a workstream timeline. This view is useful for tracking overlap and hand-off points week by week, alongside the milestones already committed to above.

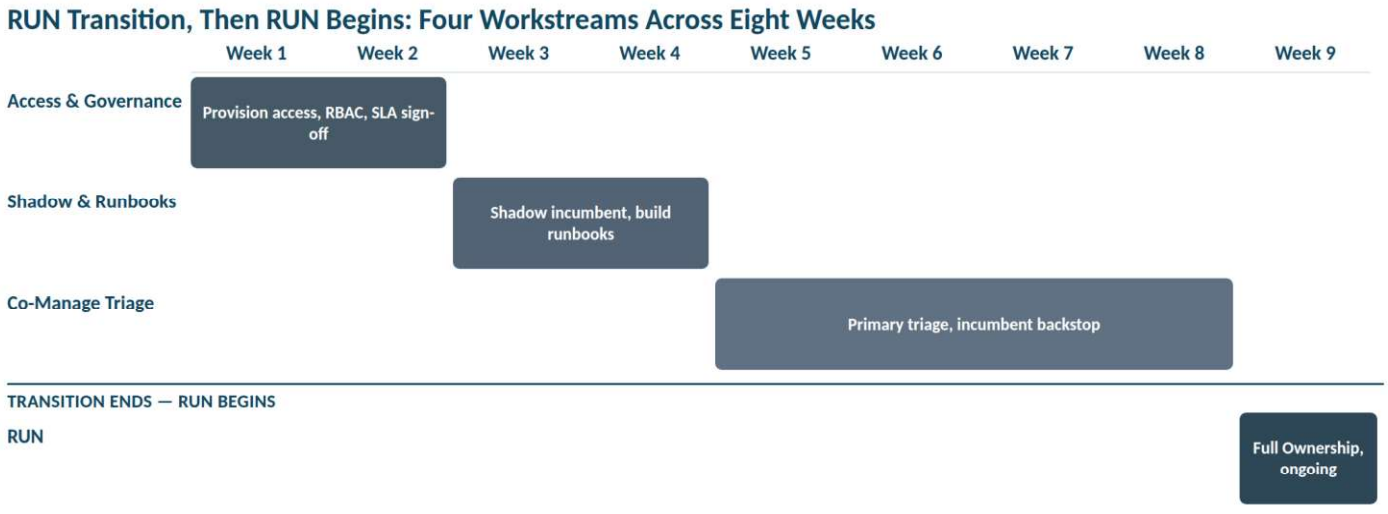


Exhibit 9. The same three Transition workstreams across Weeks 1-8, then RUN beginning at Week 9.

Change: Automation Activation Plan for Website and Mobile

Change begins on Day 1, in parallel with RUN, through a separate technical team. We propose **DevRev** as the AI triage and RCA layer: its knowledge graph organizes incidents around code-level Parts (DevRev's term for a specific service, module, or component an incident traces back to), the structure that powers the classification section 3.2 asks for and the root-cause correlation section 3.4 asks for, and its JIRA integration is genuinely bi-directional. This is not a new, unproven platform: DevRev already runs IndiGo's Payment triage and RCA today, and this extends a pattern already proven in production to website and mobile, on a new instance built the same way. Change runs in five stages. The first runs alongside RUN on a new DevRev instance; the remaining four depend on real incident data RUN generates, so they follow on once RUN is live.

Exhibit 10. The Change activation plan for website and mobile: five stages, Day 1 onward, the first running in parallel with RUN's steady-state operations.

Stage	Duration	Key Activities	Deliverable
Integrate and Instrument	Month 1 to 2	Deploy DevRev on a new instance using the pattern already proven on IndiGo's Payment triage and RCA; connect JIRA via native bi-directional sync; feed Firebase Performance Monitoring and website APM signals in; reconcile Azure Monitor, OpenTelemetry, Grafana, and Loki into one integration architecture. Runs alongside RUN through a separate technical team.	DevRev live, reading and writing ServiceNow
Build and Correlate	Month 3	Configure the knowledge graph against real incidents from RUN; convert recurring incidents into problem records and build out the Known Error Database; correlate NPS trends with production incidents; establish the CAPA process; validate data quality; secure IndiGo Digital Leadership sign-off.	Change-phase readiness confirmed
Validate	Month 4	Activate knowledge-graph triage by category, priority, and resolver group; activate RCA-to-code linkage for defect handoff; run AI recommendations in parallel against the	AI triage validated against human baseline

Stage	Duration	Key Activities	Deliverable
		human triage baseline; set confidence thresholds with IndiGo Digital Leadership.	
Automate	Month 5	Turn on no-code workflow automation for defect handoff and status sync; automate KEDB article generation from resolved tickets; bring SLA tracking and RBAC-aware AI actions fully live; measure the first month of AI-assisted MTTR against baseline.	First AI-assisted MTTR report
Scale and Govern	Month 6 onward	Full agentic AI coverage across triage, RCA, and defect handoff; continuous learning loop refines the knowledge graph with every ticket; report automation coverage and efficiency gains against the RUN's Day 1 baseline; monthly automation review with IndiGo Digital Leadership.	Automation coverage reported vs RUN's Day 1 baseline

Exhibit 11. Change stages, activities, and deliverables in detail.

Where DevRev Fits, and Where It Doesn't

DevRev is a strong, differentiated choice for AI-driven triage and RCA-to-code linkage, closer to what sections 3.2 and 3.4 actually ask for than most generic ITSM automation tools. It is not a ServiceNow replacement, an APM tool, or a source of 24x7 staffing, and we are not positioning it as any of those. The table below is deliberately explicit about that boundary.

What DevRev Is	What It Is Not
An AI triage and RCA layer that reads from and writes back to ServiceNow.	A replacement for ServiceNow. ServiceNow remains IndiGo's system of record throughout, unchanged.
A native, mature, bi-directional bridge to JIRA for engineering handoff.	An APM or observability tool. Performance signals come from Firebase Performance Monitoring and IndiGo's existing tool, integrated via webhook.
A knowledge graph linking incidents directly to code-level Parts, feeding the RCA loop.	A source of human debugging capacity. That stays a staffed commitment under RUN, with RCA staffed 16x5 and on call for P1 and P2, alongside the dedicated Change roles from Section 12.
Licensed on a seat or usage basis, tracked under the Change commercial line.	Priced as managed support. The human delivery team's cost is tracked separately under RUN.

Exhibit 12. DevRev's role in this proposal, stated explicitly in both directions.

Section 4 goes deeper on the AI capability itself, and Section 6 covers how DevRev, ServiceNow, JIRA, and the observability stack integrate technically.

Roles and Responsibilities Across RUN and Change

The RFP asks for roles and responsibilities against every phase. Rather than repeat the timelines already set out in Exhibits 7 through 14 under a separate heading, this RACI matrix maps the twelve deliverables already committed to above against who is Responsible, Accountable, Consulted, and Informed at each one.

	Service Delivery Manager	SME / Track Leads	Engineers (Triage, RCA)	Change Engineers	IndiGo Digital Leadership	IndiGo Delivery Team
Governance framework signed off	R	C	I		A	I
Runbook library v1	A	C	R		I	I
First independent P1 managed	A	I	R		C	I
24x7 ownership achieved	A	I	C		C	I
Observability stack live	A	I	I	R	C	
KEDB and problem management live	C	A	I	R	I	
Automation backlog validated with IndiGo	C	R	I	C	A	I
Change-phase readiness sign-off	R	C	I	C	A	I
DevRev live, reading and writing ServiceNow	A	I	I	R	C	
AI triage validated against human baseline	C	I	I	R	A	
First AI-assisted MTTR report	A	R	I	C	I	
Automation coverage reported vs Phase 1 baseline	R	C	I	C	A	I

R Responsible: does the work A Accountable: owns the outcome C Consulted: input sought I Informed: kept in the loop

Exhibit 13. RACI matrix for RUN and Change, mapped to the deliverables already defined in Exhibits 7 through 12.

IndiGo Digital Leadership is Accountable for every sign-off and readiness gate, never just Consulted or Informed, matching the escalation authority already established in Section 5 and Section 11. And the IndiGo Delivery Team stays Informed rather than Responsible throughout RUN and Change, reflecting the fix-ownership model stated in Section 3: we triage and diagnose, delivery deploys, unless directed otherwise. Change Engineers is a deliberately separate column from RUN's Triage and RCA Engineers: Section 12 states this is a distinct team from Month 1, so deliverables that team owns show RUN's engineers as Informed rather than Responsible, so the work is credited to the team that actually did it.

One more thing this RACI matrix does not show: as Change matures, some of these roles need fewer people to carry the same Responsible and Consulted work. Section 13 quantifies that reduction.

4. Agentic AI and ITSM Automation Capability

The RFP names agentic AI more often than any other capability in this document and treats it as the differentiator IndiGo is looking for on top of a stable operating model. This section explains exactly what that capability does, incident by incident, and which part of the RFP each piece answers. As set out in Section 3, it runs on DevRev, layered onto ServiceNow and JIRA rather than replacing either.

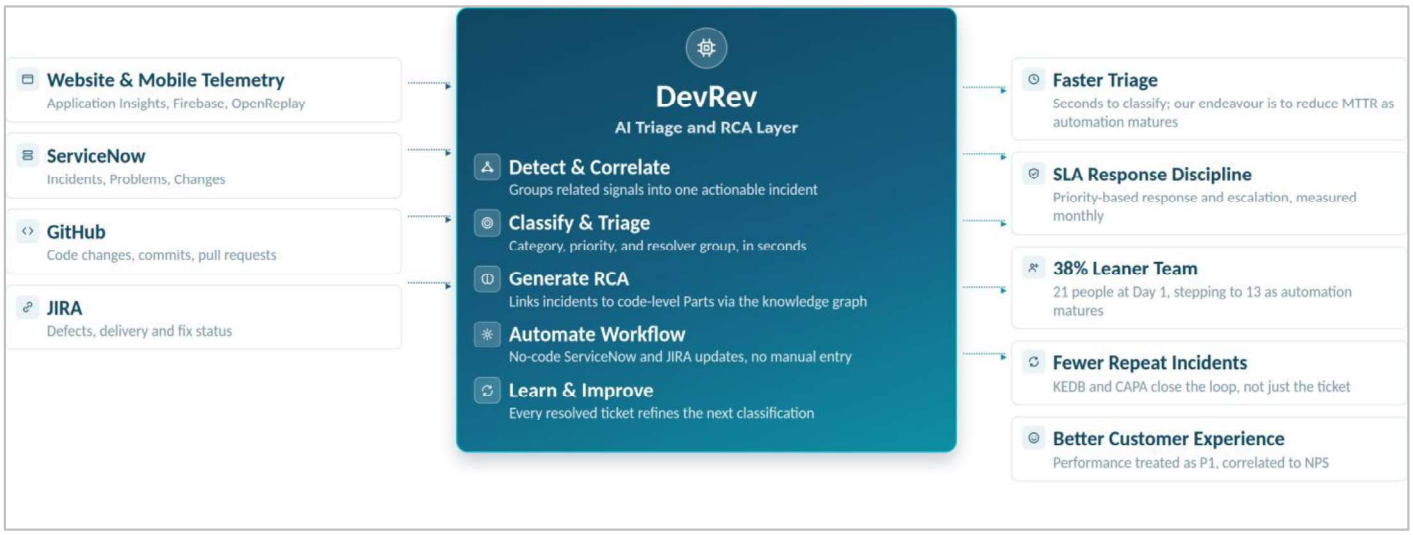


Exhibit 14. Inputs, the DevRev intelligence layer, and the outcomes each capability drives.

Mapped to the RFP's Expected Business Outcomes

The RFP names five specific business outcomes it expects this engagement to deliver. Each one is already committed elsewhere in this proposal; this table states plainly how our solution achieves each, rather than leaving the connection implicit.

RFP's Stated Outcome	How Our Solution Achieves It
Reduction in MTTR	Our endeavor is to reduce MTTR as the automation layer matures: Classify and Triage cuts time-to-assignment to seconds, and Generate RCA links every incident to its code-level Part automatically. The RFP defines MTTR as time to resolve after detection, which includes the delivery team's fix and deployment time. Our triage and RCA work compresses the front half of that clock; full resolution time also depends on delivery fix and deployment, per the fix-ownership model in Section 6.
Enhanced SLA Compliance: ≥ 99.9% service availability	IndiGo has confirmed this figure applies to our own support service's responsiveness. The website or app's uptime is a separate matter we neither host nor control. Section 5's coverage model keeps Monitoring and Triage staffed on all three shifts, 24x7x365, with a 30-minute handover overlap at every shift change. On-call RCA backs them for P1 and P2 outside RCA's 16x5

RFP's Stated Outcome	How Our Solution Achieves It
	staffed hours, so every incident gets a response from the moment it is raised. This is the coverage we stand behind for this target.
Cost Optimization through automation and resource efficiency	The direct, quantified result of the team ramp-down in Section 12: 20 people in the L2 team at Day 1, stepping to 15 once automation is live, a 25% steady-state headcount reduction that reflects automation absorbing routine work, with coverage itself staying intact.
Improved Customer Experience: faster, more reliable IT services	Section 8 treats performance degradation as seriously as an outage; both rated P1. Section 9 uses NPS data to fix the issues that actually hurt customer sentiment, not just the ones triggering the most alerts.

Exhibit 15. Every RFP-named business outcome, traced to the specific mechanism that delivers it.

How an Incident Moves Through the AI Layer

Every incident pass through the same five capabilities, in the same order, whether it is a minor mobile crash or a P1 booking outage. Each stage below maps to a specific functional requirement in the RFP.

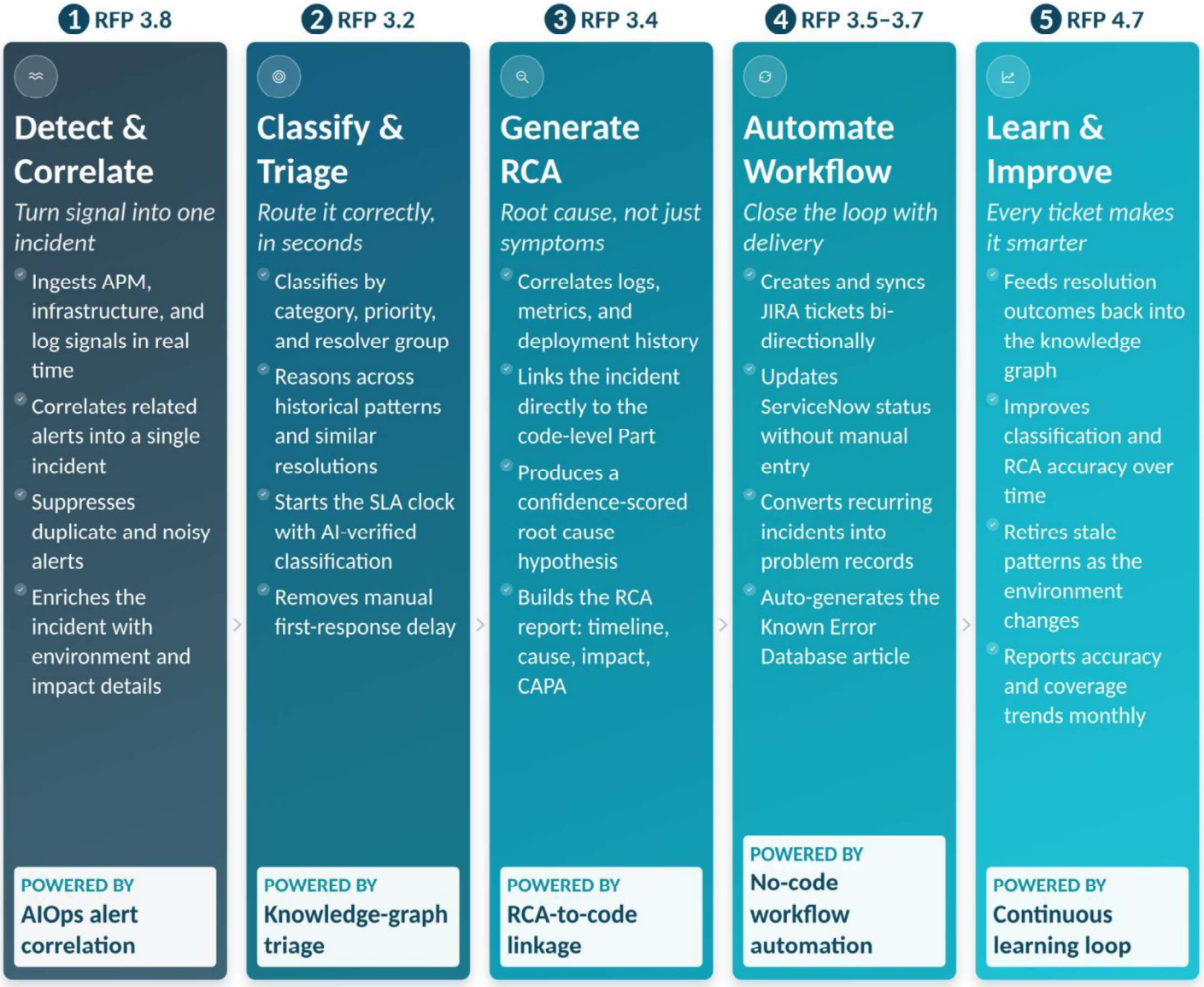


Exhibit 16. The five AI capabilities an incident passes through, mapped to the RFP's own section numbers.

Capability	RFP Ref	What It Does	How It Is Delivered
Detect and Correlate	3.8	Correlates monitoring, APM, infrastructure, and log signals into one actionable incident, suppressing duplicate and noisy alerts before they reach a human.	AIOps-style alert correlation, fed by Firebase Performance Monitoring and IndiGo's existing tool, integrated via webhook.
Classify and Triage	3.2	Assigns category, priority, and resolver group within seconds, reasoning across historical incident patterns and similar	DevRev's knowledge graph, which concerns customer, product, feature, and known-bug relationships.

Capability	RFP Ref	What It Does	How It Is Delivered
		past resolutions rather than fixed keyword rules.	
Generate RCA	3.4	Correlates logs, metrics, and deployment history into a confidence-scored root cause hypothesis, with a full RCA report: timeline, cause, impact, and corrective actions.	RCA-to-code linkage: DevRev's object model ties the incident directly to the underlying code-level Part.
Automate Workflow	3.5 to 3.7	Creates and syncs JIRA tickets bi-directionally, updates ServiceNow without manual entry, converts recurring incidents into problem records, and drafts the Known Error Database article.	No-code workflow automation, with DevRev's native Airdrop and AirSync integration to JIRA.
Learn and Improve	4.7	Feeds every resolution of outcome back into the knowledge graph, improving classification and RCA accuracy over time as the environment and incident patterns evolve.	Continuous learning loop, reported monthly against the Run's Day 1 baseline set during RUN.

Exhibit 17. Each capability, what it does, and how it is delivered, mapped directly to the RFP's functional requirements.

Explainability, Override, and Rollback

Section 4.7 of the RFP asks for AI and ML models that are explainable, continuously improved, manually overridable, and recoverable if something goes wrong. These four controls are not optional add-ons, they are built into every stage of the pipeline above, on every incident, without exception.

Exhibit 18. The four controls that keep the AI layer accountable to IndiGo Digital Leadership.

DevRev Reference Architecture

High-level reference architecture showing how IndiGo's sources flow into a single knowledge graph and how triage, RCA, workflows, analytics and stakeholder surfaces are built on that one substrate. Solid arrows are data flow; the dashed line is bidirectional write-back to Jira and ServiceNow via AirSync.

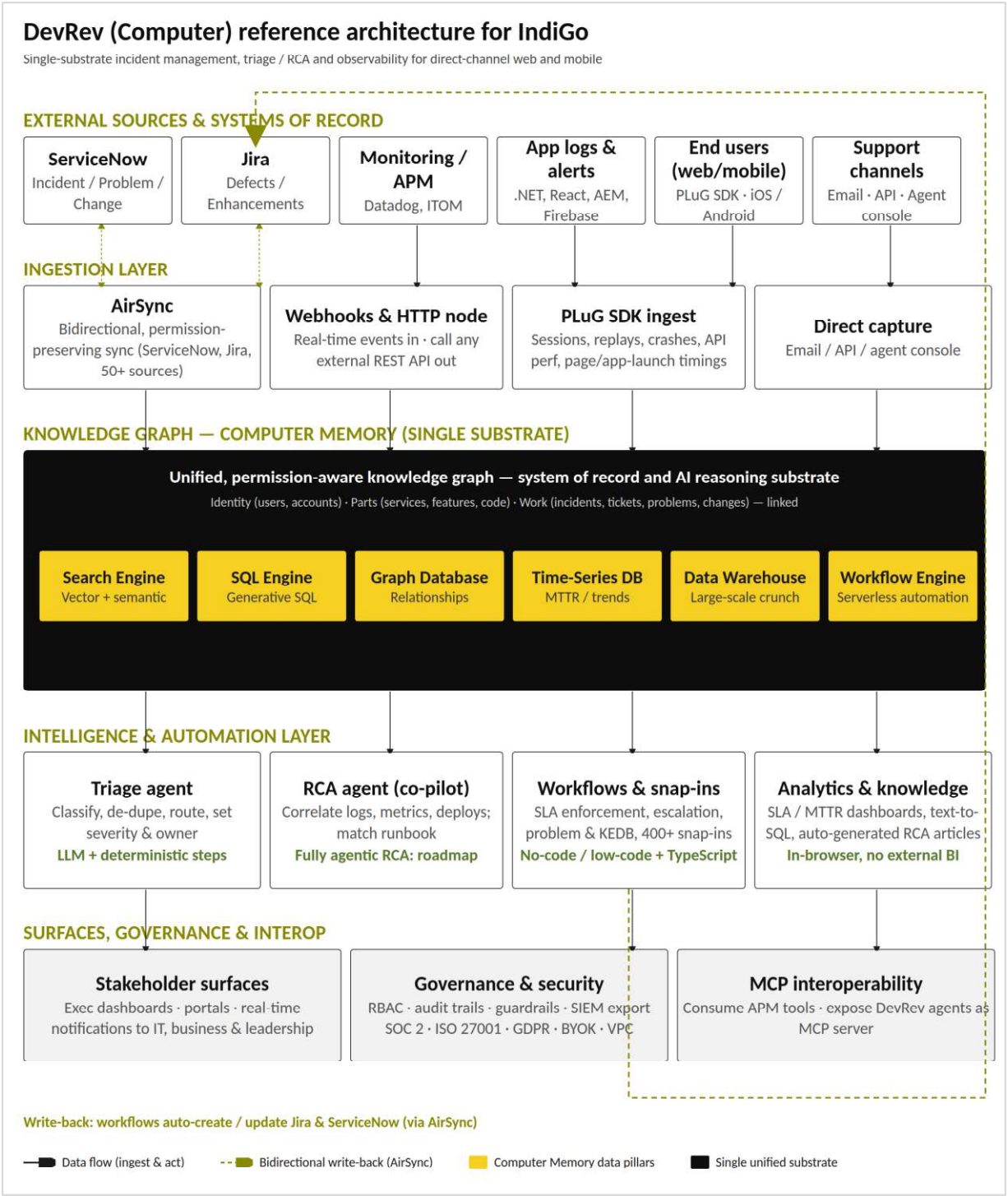


Exhibit 19. DevRev reference architecture for IndiGo: single-substrate incident management, triage/RCA, and observability across direct-channel web and mobile.

How This Works

- **External sources and systems of record:** Nothing here is new to IndiGo's stack. ServiceNow stays in the system of record for incidents, problems, and changes; Jira stays in the defect and enhancement tracker. Monitoring and APM tools, application logs, end-user sessions on web and mobile, and support channels (email, API, agent console) are the other four inputs this architecture reads from directly.
- **Ingestion layer:** Each source type reaches DevRev through whichever mechanism suits it, rather than one generic connector forced on all of them. AirSync is the bidirectional, permission-preserving sync for ServiceNow and Jira specifically, the same sync this proposal already describes in Section 6. Monitoring, APM, and application-log events arrive through webhooks and an HTTP node in real time. End-user web and mobile activity, sessions, replays, crashes, and performance timing, is captured directly through DevRev's PLuG SDK rather than inferred secondhand from ticket text. Support channel contact (email, API, agent console) is captured directly as well.
- **Knowledge graph, Computer Memory:** This is the single substrate everything above feeds into: one permission-aware graph linking identity (users, accounts), Parts (services, features, code), and work (incidents, tickets, problems, changes) together, rather than keeping them in separate systems that have to be manually cross-referenced. Underneath it sit six purpose-built engines, a search engine for vector and semantic lookup, a SQL engine for generative queries, a graph database for relationships, a time-series database for MTTR and trend data, a data warehouse for large-scale analysis, and a workflow engine for serverless automation. This is what the Parts hierarchy and RCA-to-code linkage described elsewhere in this proposal to run on.
- **Intelligence and automation layer:** Four capabilities draw on that substrate directly. The Triage agent classifies, de-duplicates, routes, and sets severity and owner, combining deterministic rules with LLM judgment. The RCA agent acts as an investigation co-pilot, correlating logs, metrics, and deployments and matching against runbooks; fully agentic RCA (resolution without a human confirming each step) is this capability's roadmap direction, still ahead of its Day-1 state. Workflows and snap-ins handle SLA enforcement, escalation, and problem/KEDB management through no-code and low-code configuration. Analytics and knowledge cover SLA and MTTR dashboards, text-to-SQL queries, and auto-generated RCA articles, all in-browsers rather than requiring a separate BI tool.
- **Surfaces, governance, and interop:** Everything above surfaces to the people who need it: executive dashboards, portals, and real-time notifications to IT, business, and leadership. Governance and security run underneath all of it, role-based access control, audit trails, and guardrails, with SOC 2, ISO 27001, GDPR, BYOK, and VPC support available as IndiGo's own security requirements demand (Section 10). MCP interoperability lets DevRev consume IndiGo's APM tooling directly and, in the other direction, exposes DevRev's own agents as an MCP server for future integration.
- **The loop closes back to the systems of record:** Workflow output does not stay inside DevRev. Automated actions write back to Jira and ServiceNow through AirSync, the same connector that brought the data in, so a ticket updated by the agent is genuinely updated in IndiGo's own systems, with no shadow copy left needing manual reconciliation.

Section 6 covers how DevRev, ServiceNow, JIRA, and the observability stack integrate technically, and Section 9 covers how problem management and RCA reduce recurrence over time.

5. 24x7 Production Support and Incident Management Operating Model

This is how production support runs, day to day, once RUN is live. It sits between Section 3 (how RUN is stood up) and Section 4 (how Change eventually automates pieces of it): the human-driven model that runs from Day 1 and continues, instrumented but not yet automated, until Change's Automate stage takes over.

Coverage Model: 24x7x365

Monitoring and Triage run three overlapping shifts around the clock, every day of the year, so no handover point is unstaffed. RCA is staffed 16x5 (Shift A and Shift B, 06:00 to 22:00, Monday to Friday), with on-call cover outside those hours for P1 and P2; P3 RCA starts in the next weekday window. This follows IndiGo's own ticket pattern (Section 12): overnight volume stays in single digits to low teens, and weekends run at a third to a half of weekday peak.

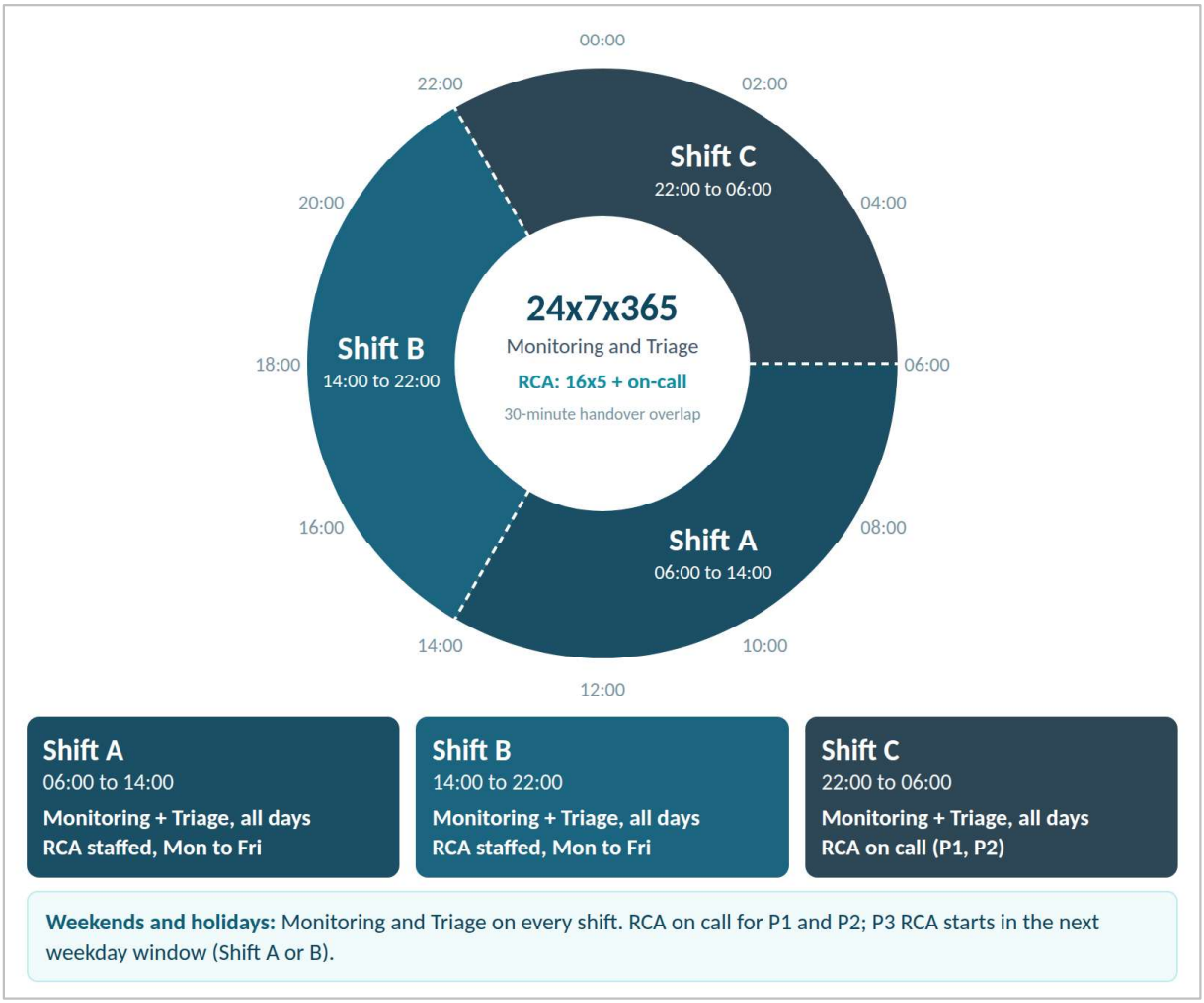


Exhibit 20. Monitoring and Triage cover all three shifts every day; RCA is staffed 16x5, with on-call cover for P1 and P2 outside those hours.

Shift	Hours (IST)	Staffing	Handover Overlap
Shift A	06:00 to 14:00	Monitoring + Triage (all days); RCA (Mon to Fri)	30 min with Shift C
Shift B	14:00 to 22:00	Monitoring + Triage (all days); RCA (Mon to Fri)	30 min with Shift A

Shift	Hours (IST)	Staffing	Handover Overlap
Shift C	22:00 to 06:00	Monitoring + Triage (all days); RCA on call for P1 and P2	30 min with Shift B
Weekends and holidays	All shifts	Monitoring + Triage; RCA on call for P1 and P2	As above

Exhibit 21. Shift staffing detail. During declared peak travel periods, on-call RCA coverage increases and the Service Delivery Manager remains on standby.

This is the Day-1 (RUN) staffing shown here. As Change's Build and Correlate stage certifies readiness and its Automate stage handles first-pass triage, Triage and RCA Engineer coverage compresses toward Shift C's lighter pattern across all three shifts as Change matures. Section 12 explains why that still holds 24x7 coverage.

The Incident Lifecycle

Every incident, any priority, moves through the same six stages, with a named owner at each one.

Exhibit 22. The six-stage incident lifecycle, role by role. Change (Section 4) augments pieces of this once the data exists to trust it.

The six stages above describe the incident lifecycle in general terms. For website and mobile tickets specifically, that lifecycle follows one concrete path today, and this is the path our integration is designed around, rather than a generic process imposed over it.

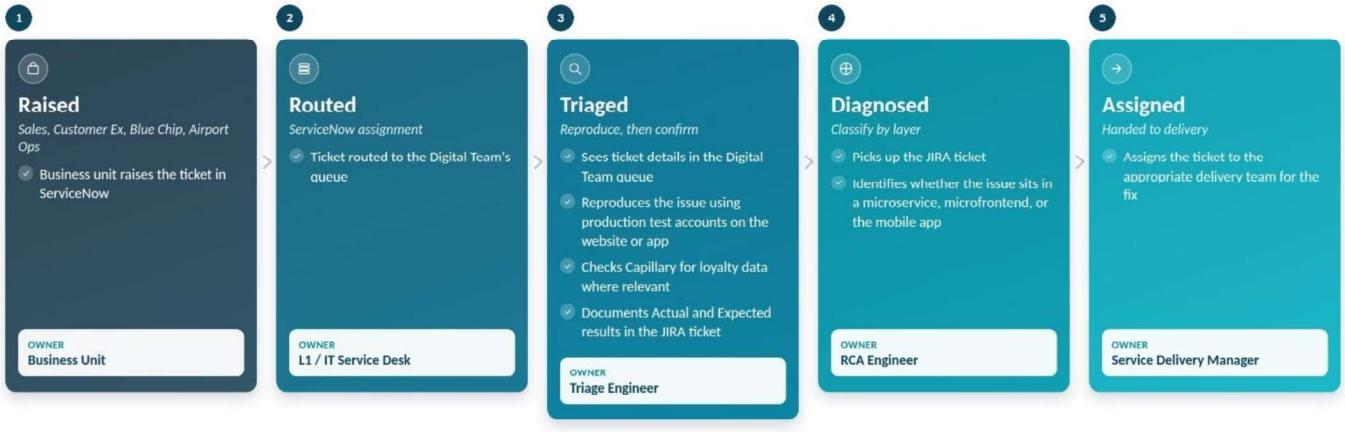


Exhibit 23. How a website or mobile ticket actually moves today, from business unit to delivery handoff.

Nothing here is a new process. Business units already raise tickets this way, ServiceNow already routes to the Digital Team, and Triage already reproduces issues on production test accounts before anything is escalated. Section 6's integration architecture is built around this same flow, including Capillary for loyalty-related tickets.

Priority-Based SLA Framework

The targets below are our proposed starting point, built from IndiGo's query clarifications and the RFP's original priority targets. Final SLA targets, the hours each SLA clock runs, and how breaches are measured will be agreed jointly with IndiGo during Transition (the Mobilize stage, Exhibit 7) and baselined before RUN begins on Day 1.

Priority	Definition	Response Time	Resolution Target	Triage Cover	RCA Cover
P1	Critical: booking/payment outage, full channel unavailability	10 min	Within 2 hours	24x7	24x7: staffed 16x5; outside these hours, on-call RCA joins the war room with the Triage Functional Analyst and Service Delivery Manager
P2	High: partial service degradation	15 min	Within 4 hours	24x7	Staffed 16x5; on call outside these hours
P3	Medium: degraded experience, workaround available	30 min	Within 12 hours (16x5 clock)	24x7	Staffed 16x5; RCA starts in the next weekday window
P4	Low: minor, cosmetic		Indigo confirmed that there are no P4 tickets.		

Exhibit 24. Proposed priority-based SLA framework and coverage by priority. Targets are indicative and will be agreed with IndiGo during Transition.

P1 War-Room Protocol

The minutes immediately after a P1 is detected are the ones that matter most. This is the same sequence every time. Out of hours, the on-call RCA engineer is paged at acknowledgement and joins the Triage Functional Analyst and Service Delivery Manager in the war room, so a P1 at 03:00 gets the same core response as one at 15:00.

Exhibit 25. The P1 response timeline, from detection to stand-down, in and out of business hours.

Operational Escalation Path

Trigger	Escalated To	Response Target
SLA response target missed	Service Delivery Manager	Immediate
P1 unresolved after 1 hour	SME and IndiGo Digital Leadership notified	Immediate
RCA/SLA priority disputed	IndiGo Digital Leadership (final authority, per Section 11)	Same business day

Exhibit 26. Operational escalation triggers, owners, and response targets.

Code-Level Debugging and Access

RCA engineers work directly in IndiGo's codebase for the affected stack, using access provisioned during RUN's Mobilize stage (Section 3). Per the stated fix-ownership model: we triage, debug, and produce a validated root cause; IndiGo's delivery team owns deployment, unless directed otherwise. Payment-related issues are reproduced using test payment instruments provided by IndiGo (Section 16).

Shift Handover and Continuity

Every shift change includes a structured handover, logged in ServiceNow, covering open incidents, pending actions, and anything P2 or above still being watched. The 30-minute overlap in Exhibit 20 exists specifically to make that handover a conversation.

Proactive Health Checks

Hourly smoke tests across web and mobile core flows, login, search, booking, and payment, published to an internal dashboard and escalated as P2 if a flow fails twice consecutively. We propose implementing this on existing Grafana observability stack for web, using IndiGo's own tool to script that same sequence on a schedule; for mobile, the equivalent check runs against the same backend APIs the app calls, since proving the path is healthy does not require driving the native app's UI. Hourly smoke testing is the RFP's own requirement; the specific flows tested, the escalation threshold, and the dashboard delivery are our own proposed implementation of it, built ahead of relying on a monitoring tool to catch it.

The Monitoring Team (Section 8) runs these checks around the clock and raises the escalation when a flow fails.

This is the model RUN runs on. Section 4 covers how Change automates pieces of it once the data exists to trust that automation; Section 11 covers the governance layer, review cadence, and liquidated damages framework this reports into.

6. Tooling, Integration and Technical Architecture

This section answers a specific, practical question: when an incident happens on the website or the mobile app, which systems talk to which, in what order, and who governs the connections between them. It maps directly to the RFP's own Architecture Checklist annexure.

Integration Architecture

Nothing in this architecture asks IndiGo to replace a system it already runs. DevRev sits alongside ServiceNow and JIRA, reading signals from observability tooling above it and writing back into the systems of record below it.

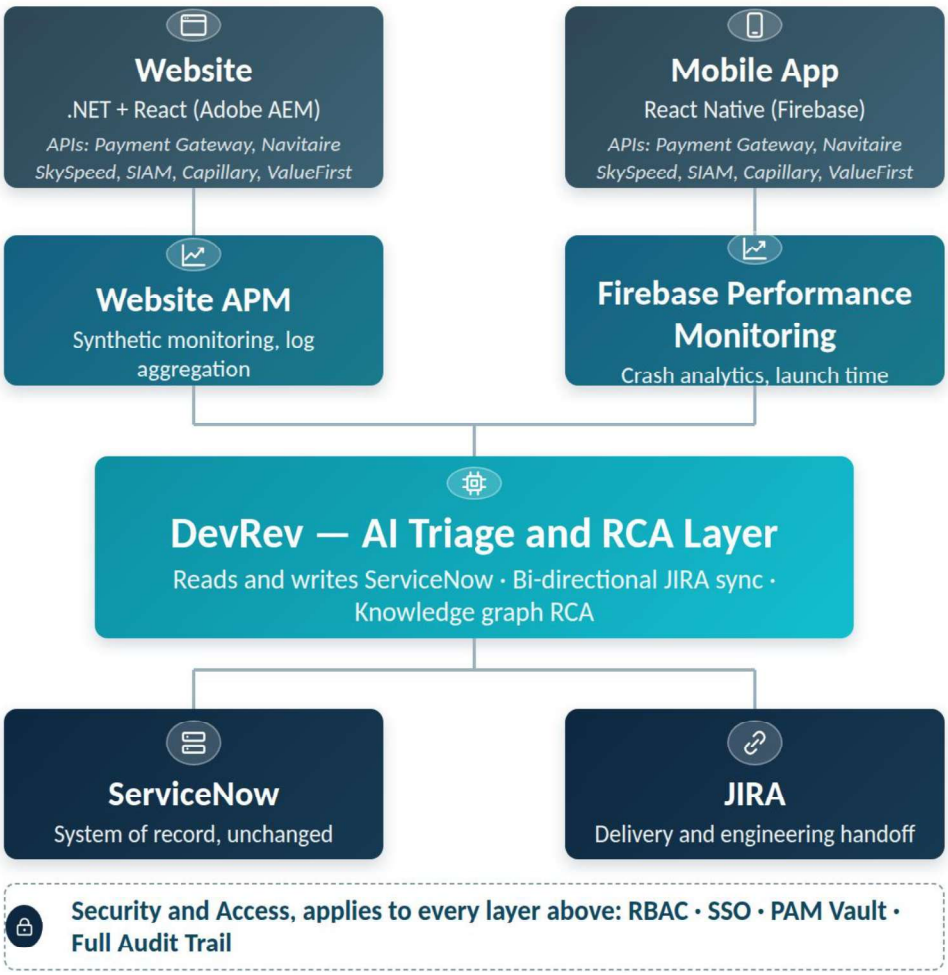


Exhibit 27. How the website, mobile app, observability tooling, AI layer, and ITSM systems connect, top to bottom.

Technology Stack at a Glance

Six layers, each doing one job. None of them overlap in responsibility, which is what keeps the architecture debuggable when something goes wrong.

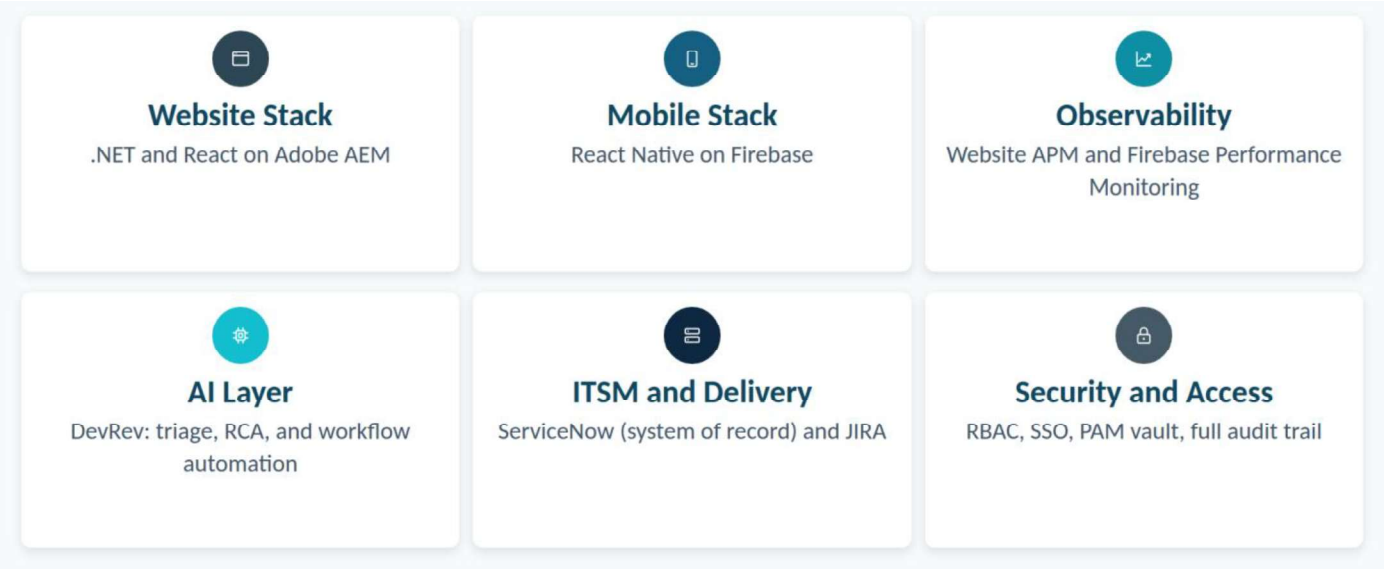


Exhibit 28. The technology stack supporting website and mobile production support, summarized.

Integration Points in Detail

Every connection in Exhibit 27 is a real, specific integration. The table below states how each one actually works and who governs it.

Integration Point	Type	How We Connect	Governance
ServiceNow	System of record	DevRev reads and writes incident and problem records via ServiceNow's native API; IndiGo's existing instance and configuration are unchanged.	RBAC-scoped service account, every write attributed and logged
JIRA	Delivery handoff	DevRev's native Airdrop and AirSync provide bi-directional sync: defects created in DevRev appear in JIRA, and status changes sync back automatically.	Delivery team retains full ownership of the JIRA workflow
Website APM	Observability	Grafana IndiGo's existing and confirmed tool, feeds alerts to DevRev via webhook. No new APM tool is proposed or required.	Alert thresholds and routing rules signed off with IndiGo
Firebase Performance Monitoring	Observability	Native Firebase instrumentation, already present in the React Native app, feeds crash and performance signals to DevRev via API.	No new SDK required; uses IndiGo's existing Firebase project

Integration Point	Type	How We Connect	Governance
Payment Gateway	Third-party API	Monitored as a dependency, not owned; incidents traced to gateway timeouts or failures are flagged with the gateway as the affected component.	No direct write access; read-only dependency mapping
Navitaire SkySpeed (PSS)	Third-party API	Monitored as an upstream dependency; RCA correlates app-level symptoms against known Navitaire SkySpeed incident windows where IndiGo shares status feeds.	No direct write access; read-only dependency mapping
IDAM	Third-party API	Monitored as an upstream dependency for authentication; incidents involving login, session, or SSO failures are traced to IDAM as the affected component.	No direct write access; read-only dependency mapping
Capillary (Loyalty)	Third-party API	Triage Engineers check Capillary directly for loyalty account and points data during triage of Blue Chip and loyalty-related tickets, per Section 5's ticketing flow.	No direct write access; read-only lookup during triage
ValueFirst (OTP Delivery)	Third-party API	Monitored as an upstream dependency for SMS-based OTP delivery; incidents involving failed verification codes are traced to ValueFirst as the affected component.	No direct write access; read-only dependency mapping

Exhibit 29. Every integration point in Exhibit 27, stated specifically enough to configure and audit.

This architecture is the basis for the RFP's Architecture Checklist annexure response. Section 8 covers the observability layer in more depth, and Section 10 covers the security and access controls introduced here.

7. Non-Functional Requirements Compliance

This engagement is scoped to L2 production support: triage, code-level debugging within the supported stack, and root cause analysis. IndiGo web and mobile delivery teams own code fixes and infrastructure.

Several of the RFP's own non-functional requirements, read literally, describe the availability, scalability, and disaster recovery of the underlying website and mobile application infrastructure, which this engagement does not host, does not own, and cannot commit uptime for. Applying an infrastructure SLA to a support service would be a commitment we could not actually stand behind, so this section separates what we commit to as the service provider from what remains the infrastructure owner's responsibility.

Scope Boundary: Support Service vs. Application Infrastructure

Security, data privacy, interoperability, AI/ML reliability, logging and audit trail, and reporting quality are covered in dedicated depth elsewhere (Sections 6, 9, and 10) and are cross-referenced here rather than repeated.

RFP Ref	Requirement	What's In Scope	What's Out of Scope	Owner
4.1	**Availability & Reliability**	24x7 Monitoring and Triage; RCA staffed 16x5 with on-call for P1 and P2 (Exhibit 20); SLA targets agreed in Transition (Exhibit 24)	Uptime, failover, and self-healing of the website/mobile application itself	IndiGo / infra owner
4.2	**Scalability & Capacity**	ServiceNow/DevRev ticket and alert volume have no hard limit; team scales via the shift model (Section 5, 13)	Horizontal/vertical scaling of application servers for traffic spikes	IndiGo / infra owner
4.3	**Performance**	Incident classification within seconds, per Exhibit 22's SLA clock	—	AIONOS, in full
4.8	**Resilience & DR**	DevRev configuration, and support history	RPO/RTO and business continuity of the application during infrastructure failure	Shared: AIONOS owns DevRev configuration and support history; IndiGo or the infrastructure owner owns the application's RPO/RTO and business continuity.
4.9	**Maintainability & Configurability**	Workflow, SLA, and routing changes in DevRev	—	AIONOS, in full
4.10	**Usability**	Role-based dashboards and support tooling UX (Section 11)	Customer-facing website/app UX	AIONOS for support tooling only

RFP Ref	Requirement	What's In Scope	What's Out of Scope	Owner
4.13	**Supportability & Service Management**	Support model, escalation, and response timelines (Section 5); coordination of ServiceNow/DevRev platform updates	Patching and updates of the application and its infrastructure	Shared: AIONOS owns support response timelines and ServiceNow/DevRev platform coordination; IndiGo or the infrastructure owner owns patching and updates of the application and its infrastructure.

Exhibit 30. Non-functional requirements split by what the support service commits to versus what the application infrastructure owner is responsible for.

This split is not a way of declining requirements. Every row above still has a specific, measurable commitment on our side; it is scoped to what an L2 triage and RCA engagement can be accountable for, rather than a blanket of acceptance of infrastructure SLAs we have no operational control over. Where a requirement is genuinely shared (Exhibit 30's 4.8 and 4.13), the governance model in Section 11 defines how the two owners coordinate rather than leaving the boundary implicit.

8. Digital Performance Management and Observability

This section responds to functional requirement 3.9. IndiGo already runs Grafana as its observability platform and has confirmed it is looking for a team to monitor its applications on that platform. This section covers that team first, then the metrics it watches, how symptoms are traced to root cause, and how detection and optimization work.

24x7 Application Monitoring Team

We propose a dedicated monitoring team of six analysts, providing round-the-clock coverage on IndiGo's existing Grafana stack. The team's job is to spot problems early, confirm they are genuine, and raise a ServiceNow ticket with enough detail for the L2 Triage and RCA teams (Section 12) to start work immediately. Exhibit 31 shows where the team sits in the incident flow.

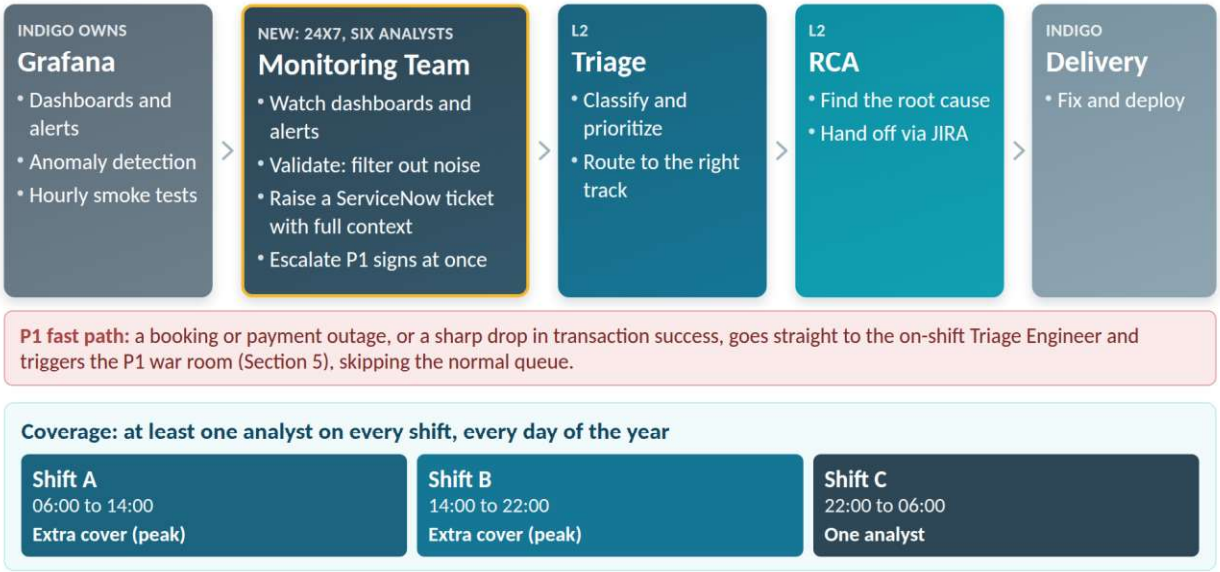


Exhibit 31. The 24x7 Monitoring Team sits between IndiGo's Grafana stack and the L2 Triage and RCA teams, with at least one analyst on every shift.

What the Monitoring Team Does

- **Watch:** Continuously monitor Grafana dashboards and alerts across the website (B2C and B2B) and the mobile app, with priority on booking, payment, check-in, and login journeys.
- **Validate:** Check every alert before raising a ticket. About 31% of the tickets in the data IndiGo shared (1,600 of 5,091) were alert noise; filtering these at the monitoring stage keeps L2 focused on genuine issues.
- **Raise:** Create a ServiceNow ticket for every confirmed abnormality, with a suggested priority.
- **Escalate:** For anything that looks like a P1, such as a booking or payment outage or a sharp drop in transaction success, alert the on-shift Triage Engineer directly and trigger the P1 war-room protocol (Section 5).
- **Report:** Run the hourly smoke tests the RFP asks for (Section 5) and publish application health at regular intervals, as the RFP also requires.

What Every Ticket Contains

Each ticket reaches L2 with what was observed and when it started; the affected journey, channel, and platform; links or screenshots from the relevant Grafana dashboard; related alerts that fired in the same window; any recent deployment or change in that window, where visible; and the estimated customer impact. A Triage Engineer can start investigating from the ticket alone, without first going back to the dashboards to work out what happened.

Coverage and Staffing

Six analysts give at least one person on every shift, 24 hours a day, every day of the year. The remaining capacity adds cover on Shift A and Shift B, where about 89% of genuine incidents arrive in IndiGo's ticket data, and covers weekly offs and leave. Six is the minimum for single-analyst coverage around the clock, so the team stays at six for the full term while the L2 roles reduce as described in Section 12. It is part of the RUN team structure shown in Exhibit 40. The team reports to the Service Delivery Manager and follows the same shift handover process as the rest of the team (Section 5).

Boundaries

Root cause analysis and fixes stay with the L2 Triage and RCA teams and IndiGo's delivery team. Grafana remains IndiGo's platform: the Monitoring Team uses it, and configuration, alert thresholds, and dashboard coverage stay with IndiGo. Where the team finds a monitoring gap, such as a journey with no dashboard or an alert that fires too often to be useful, it raises a ticket for IndiGo's platform owners.

Instrumentation: Confirmed Tools, Across Every Channel

IndiGo's confirmed tooling is Grafana for observability, and OpenReplay for session-level replay. The .NET backend is instrumented using OpenTelemetry, continuing IndiGo's own existing practice rather than introducing a new standard. Change's Integrate and Instrument stage (Section 3) is where we reconcile this against IndiGo's full observability stack, including Grafana and Loki, into one integration architecture. Because OpenTelemetry is vendor-neutral, re-pointing the exporter to a different backend later is a simple configuration change rather than a full re-instrumentation project.

Instrumentation still does not extend uniformly across the whole stack, deliberately. The React frontend uses the Application Insights JavaScript SDK for page views and user interactions, because OpenTelemetry's browser-side support is not yet the industry-mature path for this kind of telemetry. The mobile app, iOS and Android both, uses Firebase Performance Monitoring's native SDK, consistent with the Firebase integration already described in Section 6; Firebase supports both platforms natively, so no separate iOS-specific or Android-specific instrumentation is required. OpenReplay adds a fourth layer specifically for the website: session-level replay of network activity, console logs, JS errors, and application state, so an RCA engineer can see what a customer actually experienced rather than infer it from metrics alone. Four instrumentation approaches across one stack, each doing the job the others cannot, rather than one standard forced everywhere for its own sake.

Metrics Tracked, End to End

Per RFP 3.9, these are tracked continuously across the website, desktop and mobile web alike, and the mobile app on both iOS and Android, continuously rather than on a periodic sample.

Metric	Channel	Source	Treatment on Breach
Page load time	Website	Grafana	P1 if customer-facing and sustained
API response time	Website and Mobile	OpenTelemetry traces (web); Firebase (mobile)	P1 if customer-facing and sustained
App launch time	Mobile	Firebase Performance Monitoring	P1 if customer-facing and sustained
Error rate and crash frequency	Website and Mobile	Application Insights / Grafana (web); Firebase Crashlytics (mobile)	P1 regardless of duration

Exhibit 32. Performance metrics tracked continuously, with degradation treated as a priority incident in its own right, rather than sitting in a separate monitoring workflow.

Correlating Frontend Symptoms with Backend Root Cause

A page-load regression is a symptom; it still needs a diagnosis behind it. The same Detect and Correlate capability described in Section 4 traces a frontend performance symptom back to the backend service, API, or third-party dependency actually responsible, using the same Parts taxonomy from Section 3 that maps each incident to the specific code component behind it. This section does not repeat that mechanism; it states that performance incidents flow through the identical triage and RCA pipeline as any other incident; there is no separate, lesser process for performance issues.

Proactive Detection, Before Customer Impact

Grafana's own machine learning layer is what makes proactive detection possible here. It uses machine learning to learn each application's normal behavior automatically, rather than requiring manually-set thresholds, and its failure-anomaly output includes a correlation with recent deployments or updates that may have caused the issue, the same deployment-history signal Section 13 currently treats as a near-term roadmap item. A degradation trend gets flagged as it develops, not after enough users have already been affected to generate incident reports, using the same alerting pipeline the rest of this proposal relies on, applied to performance signals specifically rather than treated as a separate system. The Monitoring Team (above) acts on these alerts day to day: a Grafana anomaly becomes a validated ServiceNow ticket within the same shift.

Continuous Optimization of Customer Experience

Detection and diagnosis answer what went wrong and why. Optimization is a separate commitment: that the same issue, or the same class of issue, becomes less likely over time. Section 9’s Known Error Database and recurrence-reduction tracking apply directly to performance incidents, not only functional ones; a page-load regression that recurs on the same Part (the code component behind it) is exactly the pattern Section 9 is built to catch and close. Section 9’s NPS correlation also applies here specifically: it ranks which recurring performance issues actually move customer sentiment, so optimization effort goes to the degradation that actually costs IndiGo customers, rather than whichever one fires the most alerts. Section 4’s Learn and Improve pipeline stage closes the loop, refining classification and RCA accuracy as the environment changes, so this is a genuinely continuous process.

9. Problem Management, RCA and Continuous Improvement

Change's Build and Correlate stage (Section 3, Exhibit 11) already commits to converting recurring incidents into problem records and building out the Known Error Database. This section is where that commitment becomes a repeatable, owned process rather than a one-time setup task, and where Section 4's Learn and Improve pipeline stage gets the mechanism that actually feeds it.

From Recurring Incident to Closed Problem

The same signature appearing three or more times is the trigger. Five stages, each with a named owner, take that pattern from detection to a permanent fix that the rest of the team benefits from.

Exhibit 33. The problem management lifecycle: detection through KEDB publication, with an owner at each stage.

Known Error Database: What Gets Captured

A KEDB entry is not a copy of the incident record. It is a distinct, structured artifact designed to be found by someone investigating a different incident months later. KEDB entries are ServiceNow Problem records; DevRev drafts and enriches their content but does not store them separately.

Field	Captures	Populated By
Symptom	The observable behavior: what a user or monitor sees	RCA Engineer
Root Cause	The confirmed underlying defect, not the immediate trigger	RCA Engineer + SME

Field	Captures	Populated By
Linked Part	The code-level component, via the same Parts taxonomy as RCA-to-code linkage	DevRev, automatically
Workaround	What to do if the permanent fix isn't deployed yet	RCA Engineer
Permanent Fix Status	Corrective action merged, or still pending delivery	Service Delivery Manager
Occurrence Count	How many incidents this entry has already prevented from re-investigation	DevRev, automatically

Exhibit 34. KEDB entry structure. The Linked Part field is what makes an entry findable during a future incident.

Recurrence Reduction: How We Measure This Is Working

A closed problem record is not the finish line. Section 4's Learn and Improve stage already commits to reporting accuracy and coverage trends monthly; the same reporting cadence tracks whether the specific Parts (the code components incidents are linked to, per Section 3) with published KEDB entries actually see fewer repeat incidents afterward. A KEDB entry that doesn't reduce recurrence for its Part is a signal that the preventive action didn't close the gap, beyond simply marking a task complete.

Correlating NPS with Production Incidents

Section 3's Correlate stage already commits to linking NPS trends with production incidents. In practice, that means matching NPS timestamps and comments against incident windows for the same journey, then checking which incident types precede NPS drops, beyond simply which happen most often. That's what decides CAPA priority: a rare incident that reliably tanks NPS matters more than a frequent one customers barely notice.

10. Security, Data Privacy and Compliance

Access Control and Authentication

Role-based access control is scoped to the specific roles defined in Section 13, a concrete mapping rather than a generic policy statement. A Triage Engineer's access to production data is narrower than an RCA Engineer's, which is narrower than the Service Delivery Manager's; least privilege is enforced by role from the start, rather than granted uniformly and restricted after the fact. ServiceNow and DevRev access authenticates through IndiGo's existing SSO provider, so no separate credential set exists for this engagement tooling. Every personnel access event, sign-in, production source code access, configuration change, is logged and auditable, per RFP 4.4, feeding the same audit trail Section 11 already commits to reviewing.

Encryption and Environment Isolation

Data in transit and at rest is encrypted using industry-standard protocols throughout the architecture described in Section 6. Production and non-production environments remain isolated; the RFP's own expectation that vendor personnel access production source code (Section 3) makes this isolation a control, since the same personnel with source access must not have an ungoverned path from non-production testing into production systems.

PII Masking

Personally identifiable information is already masked in logs, dashboards, and reports before it ever reaches the AI layer, regardless of how a ticket is subsequently classified or triaged.

Data Privacy Regulations

Compliance applies to both frameworks explicitly relevant here, not GDPR alone: India's Digital Personal Data Protection Act, 2023, since IndiGo is an Indian data fiduciary handling Indian customers' data, and GDPR, given IndiGo's international passenger base. Data retention and archival periods are configurable to IndiGo's own standards rather than a fixed default we impose, consistent with RFP 4.5.

PCI-DSS Posture

Worth being precise here rather than broad. Section 6 defines the Payment Gateway explicitly as a monitored, read-only dependency: this engagement does not process, store, or transmit cardholder data. That said, incident and log data touching payment flows could incidentally surface payment-adjacent metadata, transaction identifiers, gateway error codes, and the same masking commitment above applies to that data before it reaches logs, dashboards, or DevRev's knowledge graph.

This is a limited-visibility touchpoint, not a payment-processing role, and the applicable PCI-DSS scope and SAQ type should be confirmed jointly with IndiGo's payment and compliance teams rather than assumed here. We have carried this forward as an open item in Section 16 rather than asserting a compliance level we have not yet validated against IndiGo's actual PCI environment.

11. SLA, Governance, and the Target Operating Model

Section 5 covers how production support runs day to day. Section 6 covers what it runs on. Section 12 covers who runs it. This section is where those three come together into one operating model, plus the governance layer that sits above all of them. SLA targets, measurement rules, and service credits are agreed jointly with IndiGo during Transition and reviewed at each quarterly business review.

Exhibit 35. The four pillars of the target operating model, and where each is detailed.

Delivery Location

The RUN team works offsite at AIONOS office (Gurugram & Noida). Monitoring and Triage work all three shifts (Exhibit 20), including 22:00 to 06:00; RCA works Shift A and Shift B on weekdays, with on-call cover from the same team outside those hours. Night and weekend coverage uses the same offsite team structure as day coverage, on a different roster. Access, workstation provisioning, and badge/security requirements for an offsite team are addressed under Security and Access (Section 10).

Program Governance and Escalation

Section 5's escalation path (Exhibit 26) covers incident-level triggers: an SLA missed, a P1 unresolved, an RCA or priority dispute. This is the layer above it, for issues that are about the engagement itself rather than any single incident: commercial questions, staffing changes, or a pattern of recurring problems that no single incident escalation would surface.

Trigger	Escalated To	Forum
Recurring pattern across multiple incidents	Service Delivery Manager, then IndiGo Digital Leadership if unresolved	Weekly ops review
Staffing, commercial, or contract question	IndiGo Digital Leadership	Monthly steering committee
SLA performance trend, quarter to date	IndiGo Digital Leadership	Quarterly business review

Exhibit 36. Program-level escalation, distinct from Exhibit 26's incident-level path.

Review Cadence

Four cadences, each reporting to the next. The quarterly business review is not a new commitment, it is the same QBR already named in RUN's Full Ownership stage, Change's Build and Correlate stage, and Change's Scale and Govern stage (Sections 3 and 4): this is where those individual QBRs are formalized as one recurring rhythm.

Exhibit 37. The governance rhythm: daily, weekly, monthly, and quarterly, each feeding the next.

Executive Dashboards and Reporting

IndiGo Digital Leadership gets real-time visibility into SLA compliance, and incident volume by priority, sourced from ServiceNow and the observability stack (Section 6). The monthly steering committee reviews the same dashboard the team uses day to day.

12. Team Structure and Talent Model

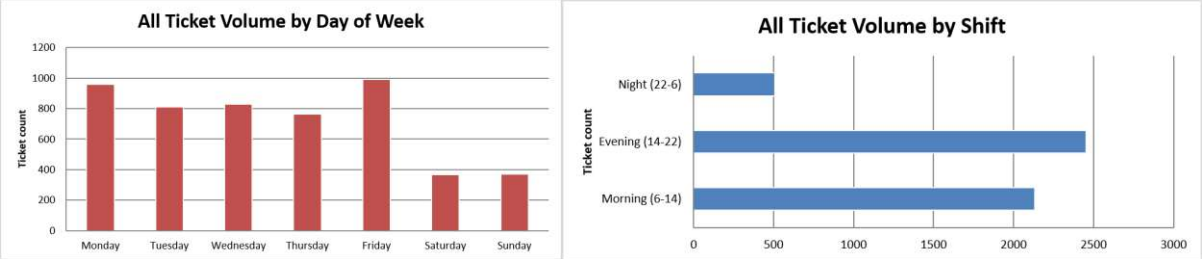
We have analyzed the ticket dump shared on various dimensions and came up with this conclusion. On an average 400 tickets are handled by triage team and approximately 100 filter tickets are passed to RCA team on monthly basis.

Heatmap: All Tickets by Day of Week x Hour of Day (noise included)

Darker = more tickets. Columns are hour-of-day (0-23); rows are day-of-week.

Day / Hour	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Monday	8	7	5	5	9	3	7	5	8	25	44	42	224	76	99	73	50	67	59	27	33	57	12	13
Tuesday	10	8	5	5	9	2	7	7	11	21	53	47	149	90	54	55	63	56	54	21	19	37	13	14
Wednesday	12	13	7	10	16	8	5	6	10	25	41	64	190	61	58	63	61	47	54	14	22	21	10	11
Thursday	8	7	5	9	10	7	6	6	14	9	41	38	97	65	66	72	51	61	81	24	30	20	14	25
Friday	22	14	6	6	5	5	8	5	7	17	47	64	147	51	57	60	90	49	40	52	26	176	24	12
Saturday	12	8	8	4	5	1	7	4	7	16	19	16	33	20	21	39	25	23	21	24	9	12	20	12
Sunday	5	4	7	3	4	2	6	5	6	19	8	12	99	15	16	39	23	15	13	18	15	20	10	8

Exhibit 38. Tickets by day of week x hour of day.



Tickets by day of week and shift in last one year

By Day of Week and Hour

- **Weekday peak:** Monday through Friday carry the clear majority of volume, concentrated in a 9 AM to 6 PM band; Monday and Friday run slightly heavier than the midweek days.
- **Weekend is real but lighter:** Saturday and Sunday run at roughly a third to a half of weekday peak volume, with the same daytime shape but a flatter, lower curve.
- **Overnight is consistently thin:** Midnight to 6 AM stays in the single digits to low teens across every day of the week, weekday or weekend.

This supports a staffing model weighted toward weekday daytime hours, with lighter weekend coverage and minimal overnight presence, rather than a flat, equal-sized shift structure around the clock. It also lines up directly with why the proposed structure below staffs 5 Triage Engineers on weekdays against 3 on weekends, and does not propose an overnight-specific headcount at all.

Heatmap: Technology / Channel x Hour of Day (All Tickets, noise included)

Darker = more tickets. Columns are hour-of-day (0-23); rows are channel.

Channel / Hour	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Mobile App	4	1	0	1	1	0	1	1	3	7	16	28	38	21	19	23	29	20	13	6	5	6	5	1
Website / WebApp	13	12	7	8	5	5	5	4	9	32	48	41	83	67	72	62	65	56	44	33	30	26	26	24
API / Backend Services	10	19	11	10	22	7	10	10	15	14	42	44	76	92	75	67	57	55	31	35	33	19	22	19
Agent / B2B Portal	1	0	0	1	0	0	1	0	1	2	20	12	23	24	16	18	16	30	10	1	2	2	1	1
Social Media	5	5	1	4	1	3	9	2	1	2	6	2	8	7	9	5	6	9	9	10	9	8	4	8
Not Specified / General	44	24	24	18	29	13	20	21	34	75	121	156	711	167	180	226	190	148	215	95	75	282	45	42

Exhibit 39. Technology / channel x hour of day.

By Channel

- **Website leads:** Website/WebApp carries the largest volume among named channels, peaking in the same 9 AM to 6 PM band.

- **Mobile App is real but smaller:** Roughly a third to a half of Website/WebApp's peak volume; present throughout the day rather than sharply concentrated.
- **API/Backend is not a minor channel:** API/Backend Services runs close behind Website in several peak hours, which matters directly for how RCA work gets split, since backend-originating incidents are not a small residual category.
- **Not Specified/General dominates the table:** This is, by a wide margin, the single largest bucket in the entire channel table, exceeding even Website/WebApp at several peak hours (over 150 tickets in some hours against Website's mid-80s).

Based on this, here is the team structure we have proposed, using the past one year's ticket data. On-call support will be drawn from within this team.

Role	Rationale
Delivery Manager	Manages the entire delivery engagement and acts as the single point of escalation and reporting to the customer.
Triage Functional Analyst	Performs first-level ticket triage. Headcount reduces Q3 as Dev Rev automation absorbs a portion of the manual triage volume.
SME (Functional)	Provides subject matter expertise to support the team on complex ticket analysis and triage decisions.
RCA Web	Performs root cause analysis for web-platform incidents. Headcount reduces Q3 as Dev Rev automation absorbs a portion of the manual RCA volume.
RCA Mob	Performs root cause analysis for mobile platform incidents. Headcount reduces Q3 as Dev Rev automation absorbs a portion of the manual RCA volume.
NPS Analysis (Functional Analyst)	Manages the recurring monthly NPS PNR analysis activity (approx. 400-500 PNRs/month; average analysis time of 30 minutes per PNR).
Dev Rev (3-5 Years)	Builds and implements Dev Rev automation ramps up team capability to drive the Triage/RCA volume reduction from Q3 onward.
Monitoring Analyst (24x7)	Watches IndiGo's Grafana dashboards and alerts around the clock and raises validated tickets for Triage (Section 8). Held at six for the full term: the minimum for one analyst on every shift with leave cover.

2. On-Call / Standby Support (covered by the core team on rotation — no incremental headcount)

Role	Coverage	Rationale
RCA Web (On-Call — Night Shift/Weekend)	On call for P1/P2	Ticket load overnight and on weekends is minimal; on-call coverage is provisioned solely to meet SLA for P1 (Critical) and P2 (High) incidents. P3 RCA starts in the next weekday window.

RCA Mob (On-Call — Night Shift/Weekend)	On call for P1/P2	Ticket load overnight and on weekends is minimal; on-call coverage is provisioned solely to meet SLA for P1 (Critical) and P2 (High) incidents. P3 RCA starts in the next weekday window.
SME (Functional) — On-Call Weekend	On call for triage & coordination	Provides weekend on-call coverage for triage support and cross-team coordination when the core team is off-shift.
Service Delivery Manager (On-Call, P1)	Joins P1 war room out of hours	Leads escalation and stakeholder updates for any P1 outside business hours, alongside the on-call RCA engineer and the on-shift Triage Functional Analyst.

Exhibit 40. Proposed RUN team by quarter, including the 24x7 Monitoring Team, with on-call support covered by rotation. Total: 26 in Q1 and Q2, stepping to 21 from Q3.

This is our proposed team for RUN: organized around how incidents actually move through monitoring, triage, and root cause analysis. IndiGo's own Grafana stack covers observability; the six-person Monitoring Team watches it around the clock and raises tickets for Triage, while IndiGo continues to own and manage the tooling itself. Change-phase roles, agentic AI and automation engineers, run as a separate team from Day 1, in parallel with RUN's steady-state operations; they are not folded into this headcount.

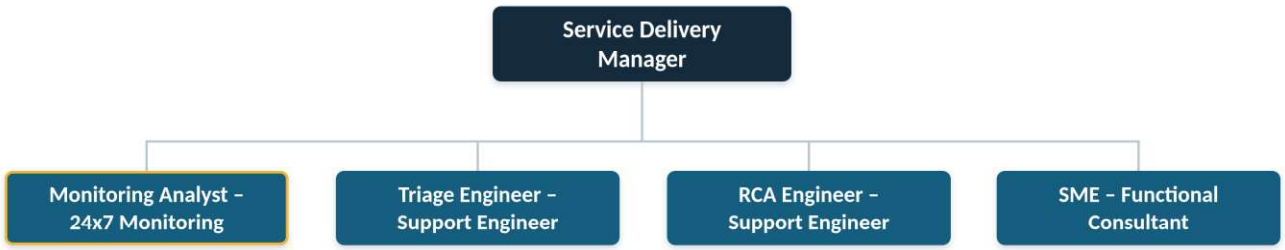


Exhibit 41. The RUN team structure.

Roles, Skills, and Experience

Track	Role	Skill Set	Experience
Delivery	**Service Delivery Manager**	ITSM project management; coordinates ticket traffic and delivery scheduling across in-scope channels.	7+ years
Delivery	**SME**	Functional consultant, cross-trained to backstop the Service Delivery Manager as the operating rhythm matures.	5 to 8 years

Track	Role	Skill Set	Experience
Triage	**Triage Engineer**	General support engineering background; web and mobile application support, first-line production incident triage.	3 to 5 years
RCA	**RCA Engineer**	General support engineering background; code-level debugging and root cause analysis across web and mobile, spanning both rather than specializing narrowly in one platform.	3 to 5 years
Monitoring	**Monitoring Analyst**	Application monitoring on Grafana; alert validation and ServiceNow ticketing; working knowledge of web and mobile booking flows.	1 to 3 years

Exhibit 42. Roles, skill sets, and experience bands for the RUN team.

Dashboard and alert monitoring is handled within RUN by the 24x7 Monitoring Team (six analysts, Section 8, Exhibit 40). That team watches IndiGo's Grafana stack and raises a validated ServiceNow ticket for anything abnormal; the L2 Triage and RCA roles above pick those tickets up the same way they handle any other incident. IndiGo continues to own and maintain the Grafana stack itself (Section 6, Section 16).

The Service Delivery Manager role is a single lead throughout RUN and Change; the two SMEs provide cross-trained backstop coverage without adding a second dedicated delivery role.

Training and enablement for IndiGo's own teams is delivered by the Service Delivery Manager and SME roles, using the knowledge base and runbooks this team builds while running RUN. Enablement happens as a byproduct of running the service, rather than as a separate workstream bolted on afterward.

Automation Potential Across the Ticket Base

We have analysed the incident data across two additional dimensions, Issue Category and AI Automation Potential, scored against 3,491 genuine tickets over 13 months (Aug 2025 to Aug 2026). This identifies specific, near-term automation candidates for DevRev, rather than treating automation as a single undifferentiated capability.

Issue Category	Genuine Tickets	Share	Automation Potential
Login, OTP & Authentication	170	4.9%	**High**
Check-in, Boarding & Seat	298	8.5%	**High**
Baggage	199	5.7%	**High**
Payment & Billing	376	10.8%	**Medium-High**

Loyalty (BluChip) & Rewards	92	2.6%	**Medium-High**
Account & Profile Management	379	10.9%	Medium
Booking, Reservation & Fare	800	22.9%	Medium
Process, Communication & Other	347	9.9%	Low-Medium
Technical / Platform (Infra & App)	830	23.8%	Low

Exhibit 43. Automation potential by issue category, 3,491 genuine tickets, Aug 2025 to Aug 2026.

Genuine tickets are all business-created incidents except those matching known automated-monitoring-alert patterns. A ticket is excluded (classified as "Noise") if its Short description contains any of the following, case-insensitive: "IBC Failure Alert(1200)," "spike alert," "high response time(121)," "about to break(131)," "4xx(44)," "5xx(74)," or "threshold(105)." These patterns represent system-generated monitoring alerts (e.g., API/backend health checks, SLA threshold breaches) rather than incidents reported by a customer, agent, or business stakeholder. Everything not matching these patterns is classified as Genuine.

The five quick-win categories below (High and Medium-High) total 1,135 tickets, 32.5% of genuine volume, roughly 87 tickets a month. The remaining categories, Account & Profile Management, Booking/Reservation/Fare, Process/Communication, and Technical/Platform, stay Medium or below: they carry more judgment calls, cross-team verification, or code-level investigation than a rules-based agent should resolve unsupervised, and are not treated as quick wins here.

Login, OTP & Authentication (170 tickets, ~13/month)

- The pattern in the ticket data is consistent: password reset requests, OTP delivery failures, and account lockouts, a large share of which currently close via “auto-closed, no user response” after a human waits and checks manually.
- **Ingest:** DevRev's AirSync connector pulls the ticket from ServiceNow the moment it is raised, already carrying the account identifier.
- **Resolve:** An Agent Studio workflow (no-code) checks the account status against IndiGo's authentication service; if the account is locked or the OTP failed to deliver, the agent triggers an unlock or OTP resend directly through the connected API, no human touch.
- **Fallback:** If the account is already healthy and the user simply has not responded, the agent applies the same wait-and-close policy that is manual today, on a timer, and logs the closure with a reason code.
- **Escalate:** Only genuine authentication-service failures, not covered by the two paths above, reach a human Triage Engineer, already carrying the diagnostic detail the agent gathered.

SAMPLE FROM THE DATA — INC4321582

Short description: *Agents are not receiving OTP and temporary password on Mails which has a domain of Outlook / Yahoo / Hotmail*

Resolution: *Incident getting Auto Closed because of no user response.*

Check-in, Boarding & Seat (298 tickets, ~23/month)

- Sampled resolutions here are largely rule-verification: confirming whether a specific SSR (special service request) applies to a specific fare or booking type, then re-triggering boarding pass or check-in generation once the rule is confirmed.
- **Ingest:** DevRev's Parts hierarchy links the ticket to the specific check-in or boarding microservice it concerns, the same linkage this proposal already uses for RCA-to-code.
- **Resolve:** For known, documented rule patterns (a specific SSR and fare combination already captured in the knowledge base), the agent verifies the PNR against the rule and re-triggers the affected step directly, without a human confirming the same rule again.
- **Fallback:** A pattern not yet in the knowledge base routes to a human with the rule-check already completed and attached, cutting investigation time even where the agent cannot close it outright.

SAMPLE FROM THE DATA — INC4357021

Short description: *Cst didn't got the boarding.*

Resolution: *The reported PNR has been reviewed and it was found that the XSAT SSR is not applied to the booking. As per the Droid's processing logic, only PNRs containing an XSAT SSR are eligible for automated check-in.*

Baggage (199 tickets, ~15/month)

- This category needs an honest caveat. Sampled resolutions frequently depend on a cross-team confirmation, Revenue Management or Product, on what the correct baggage allowance actually is for a given fare, sector, or codeshare combination. That is a judgment call an agent should not make unsupervised.
- **What the agent can close directly:** For a discrepancy matching a rule already confirmed and logged (a specific codeshare or fare-class pattern already resolved once and captured as a KEDB entry), the agent can auto-respond with that documented answer and close, without re-asking Revenue Management the same question twice.
- **What still needs a human:** For a new configuration mismatch, the agent still gathers the fare, sector, and SSR details and routes them directly into the Revenue Management or Product queue, skipping the manual triage relay, while the final answer stays a human call.
- This category is included as a quick win for the triage and routing time it saves, rather than a claim that baggage allowance disputes become fully unattended.

SAMPLE FROM THE DATA — INC4368279

Short description: *Baggage allowance is reflecting incorrect in Itinerary*

Resolution: *As per the Product team's confirmation, the baggage allowance configuration has been revised and the fix was deployed to Production under CHG0057051 and CHG0057052 on 31-Aug-2026.*

Payment & Billing (376 tickets, ~29/month, Medium-High)

- A recurring pattern: a payment gateway error spikes, gets investigated, and closes once monitoring confirms the spike has already subsided, a monitoring-confirmation pattern rather than a technical fix in most sampled cases.
- **Ingest:** DevRev's connection into the observability platform (Azure Monitor/Application Insights/Grafana), already in place for observability (Section 6), lets the agent query the payment gateway's current error rate and latency directly.
- **Resolve:** If the metrics show the spike has already returned to normal, the agent closes with the same telemetry a human would otherwise check manually, timestamped and attached.
- **Fallback:** A still-elevated error rate escalates immediately to the on-call RCA Engineer with the telemetry already pulled, rather than waiting on manual dashboard checks.

SAMPLE FROM THE DATA — INC4323873

Short description: *Payment Error*

Resolution: *This issue occurred due to the error "nsk-server:RateLimit" from the Navitaire end, which appears to be related to the ongoing outage over the past few days. We have assigned an incident to the NAV team.*

Loyalty (BluChip) & Rewards (92 tickets, ~7/month, Medium-High)

- Mostly points-balance and tier-status queries, and known correction patterns such as a missed points credit for a completed flight.
- **Resolve:** Direct API lookup against Capillary (the confirmed loyalty system, Section 6) for balance and tier status, answered without a ticket needing to sit in a human queue.
- **Known corrections:** A documented, known correction pattern, such as crediting points for a flight already confirmed as flown, triggers directly through the same API.

SAMPLE FROM THE DATA — INC2826845

Short description: *Fw: Regard bluechip update, IndiGo BluChip Membership No. 016863346*

Resolution: *For claiming IBC, Account name should exactly match with the PNR name. For redeeming IBC, Member has to login in the account and redemption can be done for Member and designated nominees only.*

Roughly 87 tickets a month, close to 30% of current Triage volume, follow a pattern specific and repeatable enough that DevRev's agent can either resolve it directly or complete the routing and diagnostic work a human currently does manually. This is the data behind the team-size reduction below.

Team Efficiency as Automation Matures

Baseline: The 26-Person Structure

This starts from the Weekday/Weekend/On-Call structure already proposed (the 20 L2 roles within the 26-person RUN team): Service Delivery Manager (1), Triage Engineer (5 weekday, 3 weekend), Functional SME (2, plus 1 on-call from that pool), RCA Fullstack Engineer (4, staffed 16x5 on weekdays, plus 1 on-call from

that pool), RCA Mobile Engineer (3, staffed 16x5 on weekdays, plus 1 on-call from that pool), NPS Analyst (2), and DevRev Automation Engineer (2). On-call coverage across every track is staffed by rotation from within this same 20-person team, not as an incremental headcount. This is the Day-1 staffing model, this reduction steps down from.

The six-person Monitoring Team is part of the RUN structure in Exhibit 40, but it stays at six for the full term, since six is the minimum for one analyst on every shift. The reduction below applies to the 20 L2 roles listed above, so total RUN headcount steps from 26 to 21.

What Automation Changes, and What It Doesn't

The automation-potential analysis identifies roughly 87 tickets a month, about 32.5% of genuine Triage volume, that DevRev can either resolve directly or substantially pre-diagnose (the five quick-win categories: Login/OTP & Authentication, Check-in/Boarding & Seat, Baggage, Payment & Billing, and Loyalty). That is the ceiling automation can realistically reach against today's ticket mix.

It is not, however, the headcount reduction. A human has to remain in the loop across every shift this team covers, including the lightest one: even where automation handles the majority of a shift's ticket volume, at least one person stays on duty to handle what the agent escalates, to supervise its actions, and to be the fallback when a pattern does not match. That floor is why the reduction below lands at roughly 25%.

The Reduction, Role by Role

This lands at Month 6, once automation is live, after the Month 5-6 pilot (Section below). Service Delivery Manager and Functional SME are unaffected: neither role's headcount was driven by ticket-handling volume in the first place, so automation absorbing ticket volume does not change their staffing logic.

Role-by-Role Rationale

- **Triage Engineer:** Weekday drops from 6 to 4: the two heaviest weekday shifts each lose one person to automation handling first-pass volume; the lightest shift was already at the one-person floor and stays there. Weekend drops from 3 to 2, with a new on-call layer of 1 added to cover what the reduced weekend shift cannot absorb directly, now that automation is live to help triage what comes in overnight.
- **RCA Fullstack Engineer:** Weekday 16x5 cover drops from 4 to 2, with each of the two staffed shifts losing one person as automation takes on first-pass RCA for the pattern-matched categories (Baggage, Payment). RCA has no weekend shift; out-of-hours on-call cover for P1 and P2 is unchanged.
- **RCA Mobile Engineer:** Weekday 16x5 cover drops from 3 to 2, as automation takes on a share of first-pass mobile RCA. Mobile starts with one fewer engineer than Fullstack, so its reduction is one person. As with Fullstack, out-of-hours cover stays on call for P1 and P2.
- Service Delivery Manager, Functional SME, NPS Analyst, and DevRev Automation Engineer unchanged: None of these four roles' staffing is driven by ticket-handling volume. The Service Delivery Manager provides oversight regardless of automation, and the SME's role backstops that oversight rather than handling first-line volume directly. The NPS Analyst runs a fixed monthly PNR-analysis workload that doesn't scale with incident ticket volume, and the DevRev Automation Engineer is building and maintaining the automation itself rather than handling incident tickets. Automation absorbing tickets does not change what any of these four roles exist to do.

IN ONE SENTENCE

32.5% of ticket volume is automatable by category, but every shift keeps a human floor regardless of volume, so the actual headcount reduction is 25%, concentrated in RCA where the on-call layer already existed, and lighter in Triage where a new on-call role had to be created to replace the shift it's reducing.

.

Rollout Timeline

The reduction above is not assumed from Day 1. It follows a three-stage timeline, each stage a gate the next has to clear:

Exhibit 44. The three-stage timeline: headcount holds through implementation and pilot, reduction lands only at go-live.

- **Month 1 to 3, Implementation:** DevRev is configured and connected across the five quick-win categories: AirSync integration, Parts hierarchy mapping, and Agent Studio workflows built and tested against historical ticket patterns, ahead of going live against real-time traffic.
- **Month 4 to 5, Pilot with Human Supervision:** Automation runs against live tickets, but every action is supervised: a human confirms each auto-resolution and auto-routing decision before it takes effect. Headcount stays at 20 through this entire stage; the pilot exists to prove the approach works, before it delivers any savings. Categories are proven in ascending order of complexity, the cleanest, most rule-based patterns first (Login/OTP and Check-in/Boarding, where the resolution logic is a fixed lookup or rule check) and the categories needing more judgment last (Baggage and Payment, where cross-team confirmation or telemetry interpretation is still part of the picture even when automated). By the end of Month 5, every quick-win category has run supervised and been validated against the human baseline.
- **Month 6, Live:** Supervision requirement lifts for the validated categories, and the reduction in the table above takes effect. The sequencing through Months 4 and 5, simplest patterns validated first, is what makes this a go-live decision grounded in demonstrated evidence per category, rather than a single unverified jump from 26 to 21 on an assumption.

This ordering, cleanest categories first, is deliberate: it means that if any category needs more time to reach a reliable confidence level, it is Baggage or Payment, the two already flagged elsewhere as needing more human judgment, not Login/OTP or Check-in, which this proposal treats as the most straightforward to automate reliably.

IndiGo wants humans only validating, not actively handling tickets, once auto-triage and auto-RCA are live. The 20-to-15 reduction above is how we get there. We're not there on Day 1 — the Month 4-5 pilot sets how fast we reach it.

13. Product Roadmap

This roadmap starts where Change's Scale and Govern stage (Section 3, Month 6 onward) leaves off. It is not a separate initiative bolted on afterward; every item below is grounded in a gap or an open question this proposal has already named elsewhere, rather than pulled from a generic vendor feature list.

One distinction worth stating plainly, since the RFP asks for it directly: everything in Sections 4 and 6 is available today, on the current version of DevRev and the architecture already proposed. Nothing in this section is a prerequisite for the commitments made earlier in this document. This is what gets built on top of a working system, distinct from what makes that system work in the first place.

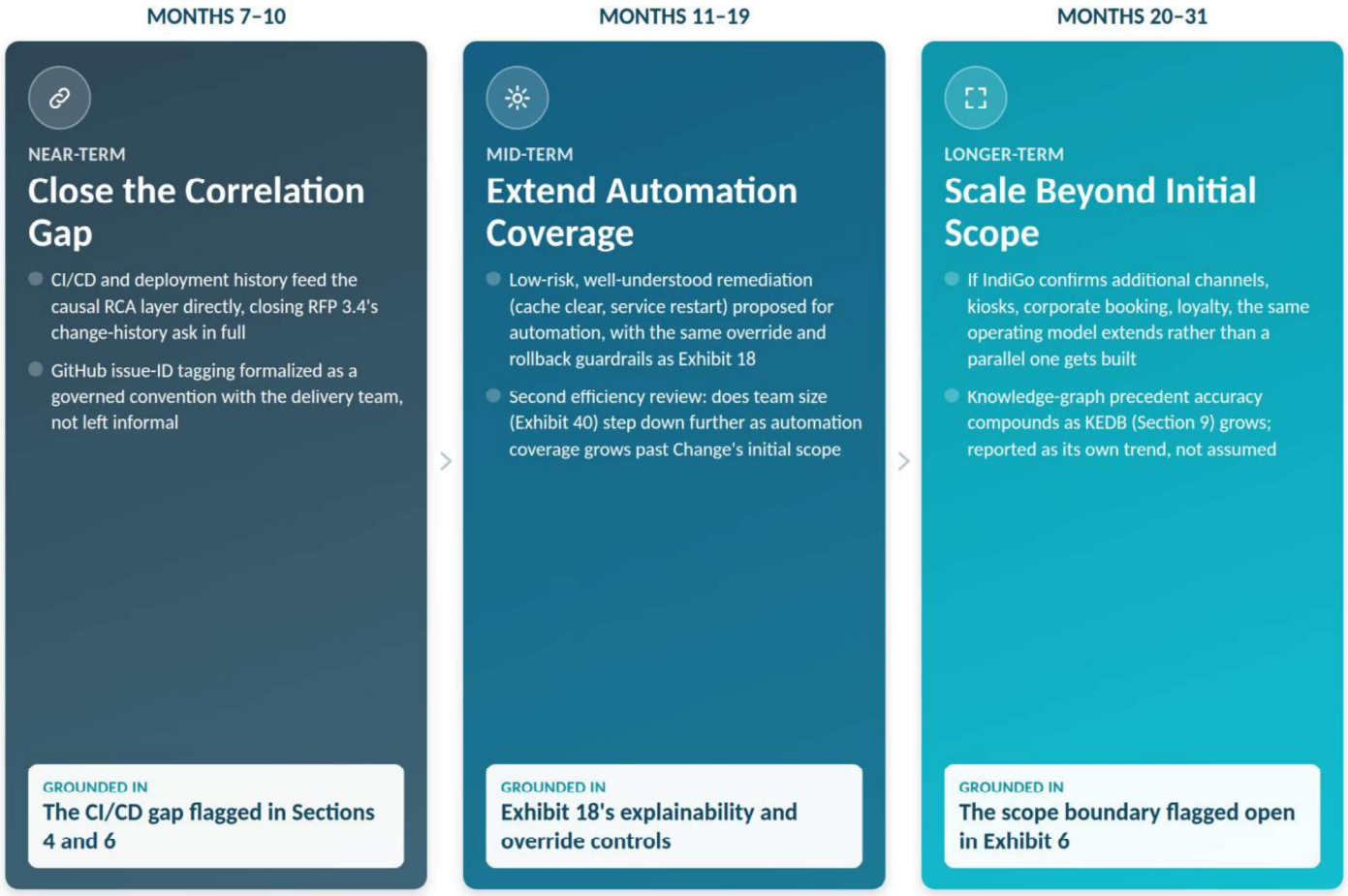


Exhibit 45. Three roadmap horizons, each grounded in a specific gap or open question already named elsewhere in this proposal.

Why These Three, and Not Others

Each horizon is sequenced by dependency. Closing the correlation gap has to come first because the mid-term automation-coverage expansion depends on trustworthy RCA input; proposing automated remediation before deployment history feeds the causal layer would mean automating on incomplete evidence. Scaling beyond the initial scope comes last because it depends on IndiGo confirming an open question (Exhibit 6) that has no answer yet; nothing technical is holding it back.

14. Relevant Experience, Case Studies and Credentials

Case Study – 1

Case Study: Driving Operational Excellence and 20% Call Deflection through IT Automation

Executive Summary

An Indian logistics provider, partnered with us for a 3-year managed services engagement to stabilize, optimize, and secure its business-critical IT ecosystem. Facing rising ticket volumes and operational bottlenecks, the engagement covered end-to-end **L2/L3 support and Change Request (CR) management** across a hybrid application suite. By deploying advanced **Service Desk Automation and AI-driven deflection tools**, we successfully transformed their IT service delivery, achieving a **20% reduction in L2 service requests** and consistently exceeding strict SLA targets.

Client Profile & Business Challenge

Indian logistics enterprise relying on a complex, interconnected digital infrastructure to manage supply chain operations, client billing, and vendor management. The client faced significant challenges in maintaining system uptime, managing high volumes of recurring service requests, and maintaining seamless data flow across disparate legacy and cloud systems. They required a strategic managed services partner to manage lifecycle operations, introduce modern ITSM governance, and drive cost-efficiencies through intelligent automation.

Scope of Work & Ecosystem

The 3-year contract established clear ownership of L2 (application support) and L3 (complex troubleshooting, bug fixes, and vendor coordination) operations, alongside continuous enhancement via Change Requests (CRs).

Technology Stack

- **Customer-Facing Portal:** Drupal-based enterprise website.
- **Core Operations:** Custom .NET billing and invoicing application.
- **Integration Layer:** Azure API Management ensuring real-time data sync.
- **Data Architecture:** Azure SQL Database.
- **Enterprise Back-Office:** Oracle Applications for procurement and financial management.

Service Management Framework

- **ITSM Governance:** Handled entirely via **ManageEngine** for ticket lifecycle tracking.
- **ITIL Processes:** Full ownership of Incident Management, Problem Management, and Service Request Fulfillment.
- **SLA Commitment:** Rigidly defined response and resolution windows for P1 (Critical), P2 (High), and P3 (Medium/Low) tickets.

The Solution: Intelligent Managed Services & Automation

Our strategy combined robust, proactive ITIL-aligned operations with a targeted automation roadmap to eliminate manual overhead.

1. Unified L2/L3 Operations

We established a dedicated support team trained across both cloud-native architectures (Azure) and enterprise suites (Oracle, .NET). Problem Management workflows were integrated to identify the root causes of recurring P1 and P2 incidents, feeding directly into the continuous improvement CR pipeline.

2. Service Desk Automation & Deflection

To address the high volume of low-complexity L2 service requests, we engineered an automation overlay integrated directly into ManageEngine:

- **Knowledge Instant-Resolve:** Implemented automated contextual search that surfaces step-by-step resolution paths to users before ticket creation.
- **Automated Service Fulfillment:** Built self-service bots to automate high-frequency, manual L2 tasks such as password resets, access provisioning, and common billing report generations within the .NET app.
- **Proactive Monitoring:** Deployed Azure-native monitoring alerts to auto-heal minor database and API connectivity glitches before they escalated into user-reported incidents.

Key Results & Business Impact

Over the course of the engagement, the operational model shifted from a reactive firefighting posture to an optimized, value-driven state.

- **20% Reduction in L2 Service Requests:** AI deflection and automated self-service successfully diverted one-fifth of incoming L2 calls, freeing the core support team to focus on critical L3 resolutions and CR delivery.
- **97% SLA Compliance:** Maintained adherence to P1 and P2 resolution metrics, minimizing operational downtime for critical logistics apps.
- **Enhanced System Reliability:** Problem management strategies reduced recurring infrastructure incidents across Azure SQL and Oracle Apps by 15%.

Case Studies – 2

Client Background

- Leading travel technology company and major distribution partner for global travel agencies across South Asia
- Connects travel buyers and sellers through a single platform; strong position in airline and aviation technology
- Serves thousands of travel agencies across hundreds of cities with distribution and booking solutions
- Mid-sized workforce; focused on modernizing travel retailing, airline content connectivity, and 24x7 support

Business Problem

- Interpreting airline fare rules and penalties (ATPCO categories 16, 31, 33, 19) was manual, slow, and error-prone
- Led to inconsistent penalty calculations for changes, cancellations, and no-shows
- Created real risk of non-compliance with airline and regulatory (DGCA) standards
- Needed automated parsing of diverse inputs (XML, JSON, text, NDC) using NLP/LLMs, reliable at scale in real-time and batch, under high concurrency

Solution by AIonOS

- Built a fare rule AI based automation solution improving efficiency, accuracy, and compliance in penalty calculation
- Identified and resolved a major latency issue in the client's Python application
- Before: 5–15 minutes per transaction for complex fare rules; ~53 hours/day of agent time on a 400-transaction desk (6+ FTEs)
- After: ~94% of rules resolved in under 3 seconds; remaining exceptions handled 40–60% faster via structured routing
- Result: overall transaction AHT cut by an estimated 70–87%, lowering costs, reducing disputes, and strengthening regulatory and airline partner confidence

Case Study – 3

Case Study — Keeping Telco BSS Always-On

Client Background

- Leading telecommunications operator running a large, complex on-premises Business Support Systems (BSS) estate
- Environment spans Production, SIT/UAT, PPT, PET, and DR, across two primary data centres
- 24x7 delivery capability split across Sydney and Pune

- Infrastructure scale: 100+ Oracle databases, ~2 PB Oracle estate, 80+ AIX LPARs, ~1,000+ VMware VMs, across 7 technology domains
- Operational volume: ~500+ Nagios alerts per month, ~157 service requests per month

Business Problem

- Customer experience, fulfilment, charging, and billing all depend on this infrastructure; keeping systems technically up was only the starting point, not the actual goal
- ~80% of monitoring alerts were informational noise, making it difficult to separate signal from noise at scale
- ~5.8 major incidents per month and ~14 formal P3–P4 incidents per month, across a genuinely large technology footprint
- The core problem was that technical severity did not reliably map to business severity: a database, middleware, or batch-scheduling issue could become a customer- or revenue-impacting event, or could be entirely inconsequential, depending on which business service actually sat behind it
- The client needed business-aware triage that understood the BSS services behind each infrastructure event, not generic technical monitoring that treated every alert as equally urgent

Solution by AIonOS

- Built an operating model combining people, process, technology, and governance around one objective: protect critical BSS infrastructure today while progressively reducing repetitive operational effort over time
- Structured delivery around a four-stage maturity model: **Operate** (24x7 run with deep domain coverage across OS, VMware, Oracle, middleware, network, storage, and Control-M), **Control** (dedicated release and deployment ownership, CAB alignment, rollback and validation discipline), **Optimize** (continuous service improvement focused on MTTR, toil elimination, capacity, and resilience), and **Intelligent** (predictive operations and intelligence-assisted first response, with human oversight retained)
- Established one operational rhythm across every shift: Detect, Triage, Assess Impact, Respond, Recover, Validate, Improve, with root cause consistently converted into a continuous service improvement and automation candidate, not just a closed ticket
- Delivered through one team spanning two locations: Sydney carrying onshore leadership and critical operations, Pune carrying specialist engineering and offshore capacity, connected through structured handovers and shared governance
- Phased a three-year automation maturity roadmap: Year 1 focused on classification and runbook-based remediation (8–10% cumulative demand retirement), Year 2 on predictive performance and patch orchestration (12–15% cumulative demand retirement), and Year 3 on agentic and LLM-assisted triage and first-response assistance (>18% cumulative demand retirement)
- Kept automation bounded by explicit controls throughout: production changes remain CAB-governed, P1/P2 remediation retains human-in-the-loop review, and automated workflows are only deployed with tested rollback in place
- Targeted outcomes: 99.96% availability, 15-minute P1 acknowledgement, P1 restoration within 2 to 12 hours depending on severity, and greater than 18% cumulative operational demand retired through automation

Beyond AIONOS's own delivery experience, DevRev's platform runs in production at scale for Paytm, HDFC Bank, ICICI Bank, Razorpay, Jio Financial Services, and Bajaj Finserv, regulated, high-transaction-volume environments with reliability requirements comparable to an airline's direct channel. DevRev has achieved (as outcomes at those customers, not a committed figure for IndiGo) resolution-time improvements of up to 95%, 40-60% operational-efficiency gains, and over 60% reduction in Mean Time to Acknowledge from agent-driven routing; these are cited as evidence of the platform's track record, separate from any SLA commitment made in this proposal.

Non-Commerical Compliance

Requirement Title	Description	IndiGo Comments / RFP Response Requirements	C	PC	NC	Provider Comments
Domain Experience – ITSM Automation	Vendor must have delivered ITSM day to day task and automation solutions for large-scale enterprises in regulated industries.	Minimum 3 case studies of ITSM exposure to BAU issues and automation with triage, RCA, and SLA-backed operations.	C			Case studies added.
AI/ML Expertise	Vendor must demonstrate hands-on experience of production support and AI-driven incident classification and RCA prediction.	Provide details of AI frameworks, algorithms, and at least 2 successful implementations.	C			DevRev's knowledge-graph classification and confidence-scored triage, described in Section 4, is the AI framework in production use throughout this proposal.
AIOps Platform Integration	Vendor must have experience integrating AIOps tools (Moogsoft, BigPanda, Dynatrace, ServiceNow AIOps).	Minimum 1 case study showing AIOps integration and RCA automation.	C			Our solution (Section 6) integrates directly with Grafana, IndiGo's confirmed tooling; no new AIOps platform is proposed.
Service Delivery Model	Describe end-to-end delivery approach including onboarding, implementation, and steady-state support.	Required	C			The two-phase RUN and Change model, covering onboarding, stabilization, and automation

						through to steady-state support, is detailed in full in Section 3.
Team Structure & Roles	Provide proposed team structure including roles for AI engineers, integration specialists, and support.	Required	C			Full proposed team structure, including AI and automation roles introduced under Change, is detailed in Section 12.
Workflow & SLA Management	Explain how workflows (triage, RCA, ticket lifecycle) are managed and monitored under SLAs.	Required	C			Triage, RCA, and ticket lifecycle workflows and their SLA enforcement are detailed in Sections 5 and 6.
Security & Data Privacy	Detailed approach for secure operations including RBAC, encryption, and compliance certifications.	Required	C			RBAC, encryption, and PII masking approach are detailed in Section 10. Certificate attached.
Scalability & Flexibility	Explain how solution scales with incident volume, user concurrency, and new integrations.	Required	C			Ticket and alert volume scaling and team scaling through the shift model are detailed in Section 7, with the

						application-infrastructure scaling boundary stated explicitly in the same section.
Training & Enablement	Outline training approach for IT Ops, business stakeholders, and admin users.	Required	C			Training and enablement approach, delivered by the Service Delivery Manager and SME roles as a byproduct of running the service, is detailed in Section 12.
Reporting & Analytics	Describe reporting capabilities for SLA adherence, RCA accuracy, and automation success metrics.	Required	C			SLA adherence, RCA accuracy, and automation success reporting are detailed in Sections 8 and 11.

15. Compliance Matrix (Appendix A)

C means fully compliant. PC means compliant with a stated, already-documented dependency or roadmap item. Every PC below cites the specific section where that dependency is already named; nothing here is a new caveat introduced for the first time in this matrix.

Functional Requirements (3.1 to 3.13)

Ref	Requirement	Status	Our Response
3.1	**Incident Intake & Capture**	**C**	The platform ingests incidents from support tickets, monitoring/observability alerts, application logs and API/email/agent-console entries. Ingestion uses three architectural

Ref	Requirement	Status	Our Response
			mechanisms: AirSync bidirectional connectors for systems of record (ServiceNow, Jira), webhook triggers that receive real-time events from any external system, and an HTTP action node that lets a workflow call any external REST API. For example, a payment-gateway 5xx spike raised by a monitoring tool auto-opens a P1 incident and pages the on-call before a customer reports it. Everything lands in a single knowledge graph, so an ingested incident is immediately linked to the affected service, customer and related work. Data capture enforces mandatory fields for impact, urgency, service and affected components. **Coverage: Available + Configurable** Section 6.
3.2	**Automated Triage & Classification**	**C**	AI agents auto-triage incoming work: they classify, de-duplicate and route to the correct team, and update owner, severity and affected components without manual intervention. Classification draws on historical incident patterns, similar resolutions and application/service dependency mapping, and improves over time. Triage is not a black box: the agent combines LLM reasoning with deterministic workflow steps, and every decision is logged with full reasoning observability, so incorrect routing can be inspected and corrected. Guardrails, versioning and role-based controls govern agent behaviour, and models are not locked in — IndiGo can choose the underlying model. **Coverage: Available** Section 4, Exhibit 16.
3.3	**Priority-Based Incident Management**	**C**	P1 to P4 definitions enforced with SLA-driven response and resolution workflows and automated escalation. The platform tracks response-time and resolution-time SLAs as distinct timers per priority, with business-hours and calendar support, and enforces them through ownership-based assignment. SLA-breach thresholds trigger workflow actions — automated escalation, reassignment and notification — so a P1 that is approaching or has crossed its target escalates without manual chasing. Priority definitions (P1 booking/payment outage through P4) are configured to IndiGo's business-impact model. SLA targets are agreed with IndiGo during Transition (Section 5).

Ref	Requirement	Status	Our Response
			Coverage: Available + Configurable Section 5, Exhibit 24.
3.4	**Root Cause Analysis**	**C**	The triage agent correlates metrics, logs, deploy markers and owner tags to hypothesize a root cause and determine blast radius, matches the failure to a runbook and records every action on the incident timeline. It uses historical incidents to suggest proven fixes and generate RCAs. Fully agent-powered RCA across metrics, events, logs, traces and sessions is present in the agent acts as an RCA co-pilot. **Coverage: Available (co-pilot)** Section 13.
3.5	**Problem Management**	**C**	Incidents and problems are native object types, and problems, incidents, changes and releases are linked natively in the knowledge graph. Predictive clustering identifies recurring patterns automatically and groups related incidents so they can be promoted to a problem record. The Known Error Database is a configured view over these native objects — known errors, workarounds and resolutions — rather than a from-scratch build, and repeat-incident reduction is measured over time through analytics. **Coverage: Available + Configurable** Section 9.
3.6	**ITSM Workflow Automation**	**C**	The serverless no-code/low-code workflow engine automates the incident lifecycle, problem/RCA workflows and task assignment/tracking. Workflows are configurable without code changes, and audit logging records all actions and updates. **Coverage: Available + Configurable** Sections 4 and 6.
3.7	**Integration with Delivery & Engineering Teams**	**C**	Support, incident and engineering work share one platform and data model, so teams see real-time engineering updates, deployment logs, assignment and resolution against each incident. Post-deployment validation workflows are configurable. Integration is handled by AirSync, DevRev's patented bidirectional sync engine, which uses LLM-assisted schema mapping to transform each source system into the unified DevRev schema while preserving

Ref	Requirement	Status	Our Response
			that system's own permissions. It runs in periodic and real-time modes and supports 50+ sources including ServiceNow and Jira. Jira sync is native and productized; ServiceNow is a supported bidirectional connector for incident/problem/change. Where a specific field or flow is not covered out of the box, it is extended with a snap-in (a TypeScript serverless extension) rather than external middleware — there are 400+ snap-ins in the repository, including a bidirectional PagerDuty integration, and each becomes a reusable node in the workflow engine. Defects identified during RCA auto-create Jira tickets with status syncing back, so ServiceNow and Jira remain systems of record during rollout with no data-migration big bang. Migration at scale is proven: in one reference, 16,000 tickets, 72,000 comments and 17,000 attachments were migrated within about six hours, which de-risks the transition from the incumbent tooling. **Coverage: Available (Jira) + Configurable (ServiceNow, via AirSync/snap-ins)** Section 6.
3.8	**Monitoring & Alert Correlation**	**C**	Log-based and monitoring alerts are clustered to auto-create potential incidents; semantic analysis correlates and de-duplicates alerts, suppressing noise and preventing duplicate tickets. A monitoring webhook opens an incident and invokes the triage agent automatically. **Coverage: Available** Section 16.
3.9	**Website and Mobile App Performance Management**	**C**	DevRev correlates session replay, network logs, error codes, API failures, severity and user sentiment; it can file a ticket, set severity and map SLAs. Degradation signals can trigger workflows to engineering or CX, and critical degradation can be configured to open a P1 incident with RCA. **Coverage: Available + Configurable** Section 8.
3.10	**Stakeholder Communication & Status Management**	**C**	Real-time notifications and a stakeholder portal provide consistent visibility into incident status, actions and next steps; critical updates can be sent to all impacted customers at once. Notifications for

Ref	Requirement	Status	Our Response
			creation, status change, SLA breach and resolution are configured on the workflow engine, with executive-level dashboards. **Coverage: Available + Configurable** Section 11.
3.11	**Reporting & Analytics**	**C**	The analytics layer provides real-time in-browser reporting with filtering, drill-down (application, severity, root-cause category) and CSV export, without a third-party BI tool. It is backed by a time-series data store that keeps the full change history of every object, so MTTR/MTTD, incident-trend and recurrence metrics are computed over time rather than point-in-time snapshots. SLA, RCA-effectiveness and automation-rate views are delivered through configurable dashboards, and natural-language reporting (text-to-SQL and text-to-visualization) lets non-technical users generate views directly. **Coverage: Available + Configurable** Section 11.
3.12	**Knowledge Management & Continuous Improvement**	**C**	Auto-generated knowledge articles from resolved incidents feed the same Known Error Database as problem management. The platform auto-creates knowledge base articles from successful resolutions and recommends known solutions, surfacing similar past tickets and relevant articles when an engineer opens an incident. **Coverage: Available** Section 9.
3.13	**Governance & Audit Readiness**	**C**	Complete audit trail of incident actions, RCA approvals, and access events, aligned to IndiGo's internal governance standards. The platform provides role-based access with dynamic, context-aware permissions applied across all objects and agents; audit logging of incident actions, RCA approvals and SLA measurements; and compliance with SOC 2 (Type 1 and 2), ISO 27001:2022 and GDPR. Permissions are preserved from each source system through AirSync rather than re-implemented, so a user only ever sees the intersection of what DevRev and the source system allow. Full event and LLM logs can be piped to IndiGo's SIEM. Data is encrypted at rest (AES-256) with optional customer-managed keys (BYOK) and multi-

Ref	Requirement	Status	Our Response
			region deployment, including VPC deployment, for data-residency needs. Session data supports source-level PII masking. **Coverage: Available** Sections 10 and 11.

Exhibit 46. Functional requirements compliance, 3.1 through 3.13.

Non-Functional Requirements (4.1 to 4.13)

These carry forward the scope split from Section 7: PC here specifically means the support-service commitment is compliant while the application-infrastructure half of the same requirement sits with IndiGo or the infrastructure owner.

Ref	Requirement	Status	Our Response
4.1	**Availability & Reliability**	**PC**	Monitoring and Triage run 24x7x365. RCA is available 24x7: staffed 16x5, with on-call cover for P1 and P2 outside those hours; P3 RCA starts in the next weekday window. SLA targets and clock hours are agreed with IndiGo during Transition. Application uptime, failover, and self-healing belong to the infrastructure owner. Sections 5 and 7, Exhibit 30.
4.2	**Scalability & Capacity**	**PC**	Ticket and alert volume has no hard limit and the team scales via the shift model. Application server scaling for traffic spikes is out of scope. Section 7, Exhibit 30.
4.3	**Performance**	**C**	Incident classification within seconds of creation, per the SLA clock. Sections 5 and 7.
4.4	**Security**	**C**	Role-based access enforced by the specific roles in Section 13, SSO via IndiGo's existing provider, full audit logging of vendor personnel access. Section 10.
4.5	**Data Privacy & Compliance**	**PC**	GDPR and India's DPDP Act compliance, PII masking upstream of the AI layer, are fully addressed. PCI-DSS scope for the Payment Gateway touchpoint remains an open item pending validation with IndiGo. Sections 10 and 16.
4.6	**Interoperability & Integration**	**C**	Native integration with ServiceNow, JIRA, and the observability stack, with standard APIs for onboarding new tools. Section 6.

Ref	Requirement	Status	Our Response
4.7	**AI/ML & Automation Reliability**	C	Every AI decision carries a visible confidence score, with manual override, rollback, and a defined fallback to manual triage. Section 4, Exhibit 18.
4.8	**Resilience & Disaster Recovery**	PC	DevRev configuration is compliant. Application-level RPO/RTO and infrastructure failover sit with the infrastructure owner. Section 7, Exhibit 30.
4.9	**Maintainability & Configurability**	C	Workflow, SLA, and routing changes are managed in ServiceNow and DevRev without custom code. Section 7.
4.10	**Usability & User Experience**	C	Role-based dashboards and support tooling UX for Triage and RCA Engineers, Operations, and Leadership. Section 7.
4.11	**Logging, Monitoring & Audit Trail**	C	Comprehensive logging of system activity and user actions, with audit reports available on demand. Sections 10 and 11.
4.12	**Reporting & Analytics Quality**	C	Accurate, consistent reporting with historical trend analysis, sourced directly from ServiceNow and DevRev rather than manually compiled. Section 11.
4.13	**Supportability & Service Management**	PC	Support model, escalation, and response timelines are fully compliant. Patching and updates of the application and its infrastructure sit with the infrastructure owner. Section 7, Exhibit 30.

Exhibit 47. Non-functional requirements compliance, 4.1 through 4.13, cross-referenced to Section 7's scope split.

Six of the twenty-six requirements above carry a PC rather than a C. Every one of them points to a dependency already named in this document, either a roadmap item (Section 13), an open assumption pending IndiGo's confirmation (Section 16), or a scope boundary we set deliberately rather than left implicit (Section 7).

16. Assumptions and Exceptions (Appendix B)

Per the RFP's own terms section, assumptions and exceptions are listed separately below. Every item here has already been named as an open question, a working estimate, or a deliberate interpretation somewhere earlier in this proposal; nothing here is introduced for the first time in this section.

Assumptions

Scope and timeline:

- "Direct channel, not limited to website and app" is scoped to website (B2B and B2C) and mobile app (B2B and B2C) specifically throughout this proposal. Kiosks, cargo, and loyalty program surfaces are treated as out of scope unless IndiGo confirms otherwise.

Baseline and volumetric:

- 350 tickets are triaged monthly, with 100 tickets forwarded for RCA.
- The headcount reduction in Section 12 assumes this volume stays broadly static through Month 6. If ticket volume increases, whether from new features, product enhancements, or expanded digital channels, the reduction may not materialize on the same timeline; automation would then be absorbing a larger volume rather than shrinking the team by the same margin.
- IndiGo's confirmed tooling: Grafana for application performance monitoring, Grafana for dashboards and metrics, OpenTelemetry for instrumentation, ServiceNow as the primary ITSM tool, and OpenReplay for website session replay during troubleshooting. IndiGo has also indicated Loki is in use; Change's Integrate and Instrument stage (Section 3) is where we reconcile the complete observability stack into one integration architecture. DevRev's ingestion and knowledge-graph population depend on that observability layer being functional and actively generating signal; this proposal assumes the layer works as confirmed, not that AIONOS builds, repairs, or takes ownership of it.
- The Monitoring Team works on IndiGo's existing Grafana dashboards and alerts. Configuring and maintaining Grafana, including alert thresholds and dashboard coverage, stays with IndiGo; the team raises a ticket when it finds a monitoring gap.

Technical and architecture:

- CI/CD and deployment-history integration into the causal RCA layer is a near-term roadmap item (Section 13), still ahead of Day 1 availability.
- RCA-to-code linkage quality (Section 6) depends on IndiGo's delivery team adopting an issue-ID tagging convention in branch names, commits, or pull requests; this is a governance dependency rather than a technical guarantee.
- The DevRev confidence threshold that determines whether a ServiceNow ticket arrives pre-classified (Section 6) is not yet set to a specific numeric value; it will be calibrated during Change's Validate stage against observed classification accuracy.
- Defects in features, pages, or content components authored and configured within AEM itself, as distinct from the React micro-frontend application code that runs on top of it, are assumed out of scope; we raise and coordinate these as incidents when they present, but resolution(triage + RCA) ownership sits with IndiGo's AEM content/platform team, outside this engagement.
- We assume IndiGo will provide the team with access to an end-to-end traceability tool (spanning ticket, code, and deployment lineage) from the start of the transition phase, ahead of formal handoff from the incumbent. This is a precondition for the Shadow and Absorb stage (Section 3) to build accurate runbooks and service-dependency maps against complete historical context, rather than reconstructing it after the incumbent has stood down.
- Proactive monitoring of IndiGo's Grafana dashboards and alerts is in scope, delivered by the 24x7 Monitoring Team (Section 8), which raises a ServiceNow ticket for every confirmed abnormality. Operating, configuring, and maintaining the monitoring tools themselves remains with IndiGo. The L2 team's responsibility begins once a ticket is raised, whether by the Monitoring Team, an alert, or a user.

Team and commercial:

- Change-phase team sizing (Section 13's roadmap, AI and automation engineering roles) is a working estimate pending confirmation of final Change scope; it is not part of the 26-person RUN headcount (Section 13).

- The 24x7 Monitoring Team (six analysts, Section 8) is part of the RUN team (Exhibit 40) and stays at six for the full term.
- Delivery is assumed offsite from AIONOS's Gurugram and Noida offices. Monitoring and Triage cover all three shifts, including 22:00 to 06:00; RCA is staffed 16x5 with on-call cover for P1 and P2 (Section 5). Offshore or follow-the-sun delivery has not been confirmed as acceptable.
- SLA targets in Exhibit 24 are indicative. Final targets, the hours each SLA clock runs, and the measurement method will be agreed jointly with IndiGo during Transition and baselined before Day 1. Because P3 RCA is staffed 16x5, P3 RCA clocks are assumed to run in weekday staffed hours only. Liquidated damages are assumed to apply to the SLAs as agreed in Transition.
- The Performance Bank Guarantee clause is assumed inapplicable to this engagement, since the RFP frames it around civil and implementation contracts, and this is a software and services engagement.
- IndiGo will provide the payment instruments the Triage and RCA teams need to reproduce and test payment-related issues, including debit and credit cards, wallets, UPI IDs, and net banking access, along with any funding they require. Where the payment gateway offers a sandbox or test mode, testing will use it. Where an issue can only be reproduced in production, IndiGo will provide dedicated instruments for that purpose and arrange the refund or reversal of any resulting transactions. AIONOS personnel will not use personal or AIONOS-owned payment instruments, and any card data will be handled under IndiGo’s PCI-DSS controls.

Governance:

- Where the RFP names both Digital Leadership and Delivery as having authority over disputed RCA conclusions or SLA priority classification, we assume Digital Leadership holds final authority, consistent with the RACI in Section 3.
- The compliance response in Section 15 is built as a full line-by-line matrix against every functional and non-functional requirement, not only the RFP's own ten-row vendor capability table, on the assumption that the fuller response is the more useful one regardless of which was strictly expected.

Exceptions

- The RFP states a 40 percent MTTR improvement target described as penalty-free (Section 1), while separately defining hard resolution-time SLAs with liquidated damages by priority (Section 11). We have treated these as two distinct commitments rather than reconciling them into one figure, since the RFP's own text does not reconcile them: the MTTR figure is tracked as an improvement trajectory, and the priority-based resolution times are the contractual, penalty-bearing targets.
- IndiGo has confirmed the ≥99.9% service availability target refers specifically to our own support service's responsiveness, separate from application uptime. We treat this as a commitment tied to Section 5's shift coverage model, something within our control rather than an infrastructure metric.
- The RFP asks for 24x7 coverage of triage and RCA. Triage and P1/P2 RCA are covered 24x7; P3 RCA is delivered within 16x5 staffed hours.
- We propose that AIONOS's underlying automation platform, reusable models, and generic tooling be carved out as “Pre-existing IP” under the RFP's IP ownership clause, with IndiGo owning only IndiGo-specific configurations, data, and insights generated under this engagement. The RFP's current language does not distinguish a bespoke build from a reusable platform, and this carve-out is necessary for us to propose a platform-based solution rather than a fully bespoke one.

- We have interpreted the RFP's “no subcontracting” clause as applying to labor, separate from licensed third-party software such as DevRev or the selected APM tool. A literal reading covering licensed software would make delivering any modern ITSM automation solution structurally impossible, since no vendor builds every layer of the stack in-house.

17. Commercial Proposal

S. No.	Core Services Scope	Commercials (INR Cr) – 3 Years
1	Core ITSM Services	[redacted]
	Total Core Services Value	[redacted]
Included Scope:		
	• Triage Support (24x7 manned support)	
	• RCA Support (16x5 support with on-call coverage)	
	• DevRev (or similar internal/external) Automation Tool Cost	
	• Observability (24x7 support)	